7. Object Oriented Modeling

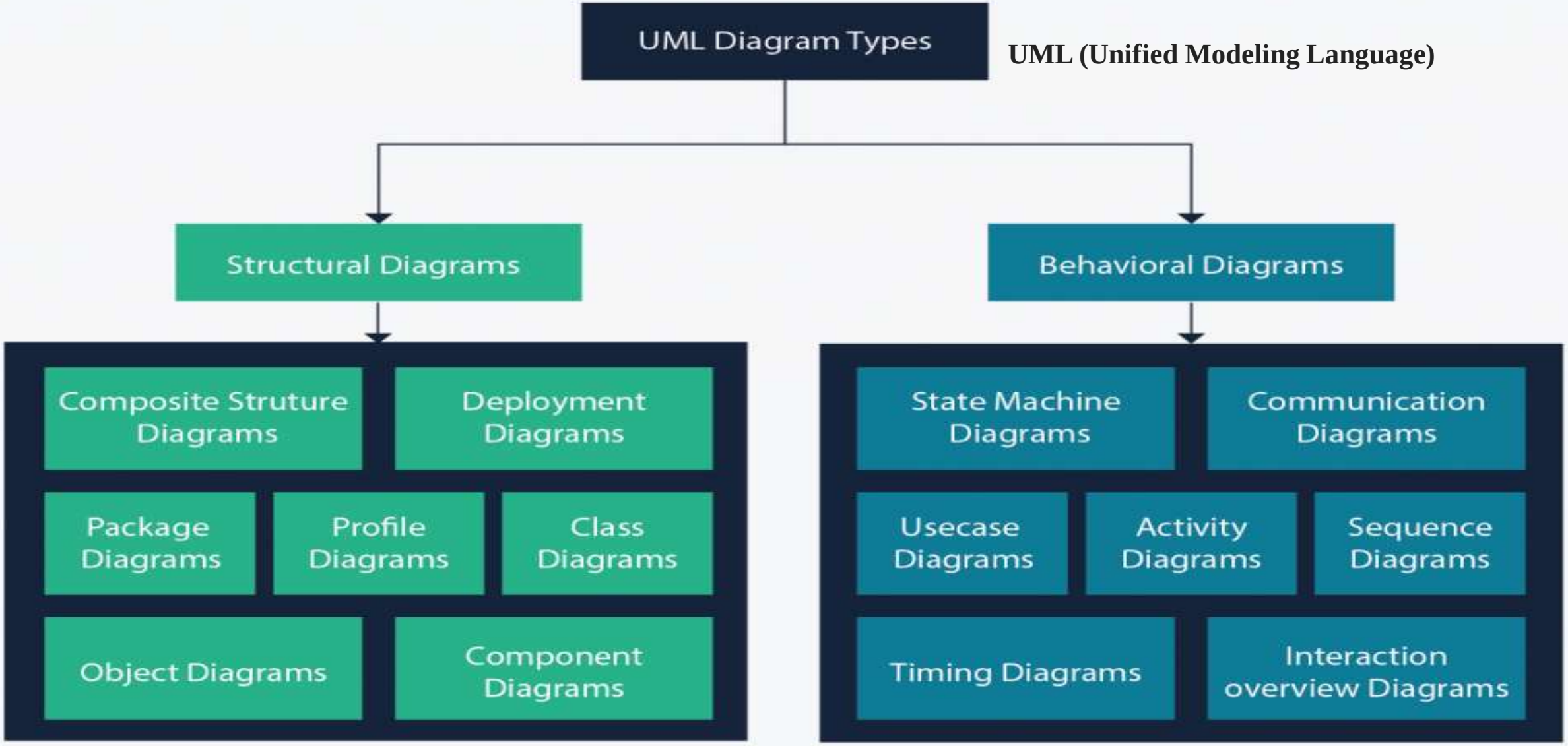
CSE333:Software Engineering

Abdus Sattar
Assistant Professor
Department of Computer Science and Engineering
Daffodil International University
Email: abdus.cse@diu.edu.bd

Learning Goals

- **Understand what object-oriented systems analysis and design is and appreciate its usefulness.**
- **Comprehend the concepts of unified modeling language (UML), the standard approach for modeling a system in the object-oriented world.**
- **Apply the steps used in UML to break down the system into a use case model and then a class model.**
- **Diagram systems with the UML toolset so they can be described and properly designed.**
- **Document and communicate the newly modeled object-oriented system to users and other analysts.**

UML Diagram Types



UML (Unified Modeling Language)

Diagram Types

- A **class diagram** represents a static view of the system. It describes the attributes and operations of classes.
- **Class diagrams** are the most widely used modeling diagram for object-oriented systems because they can be directly mapped with object-oriented languages.
- User, Customer, Administrator, Order, OrderDetails are classes. Each class consist of attributes and methods. Attributes describe the properties while methods describe the behaviors or operations.

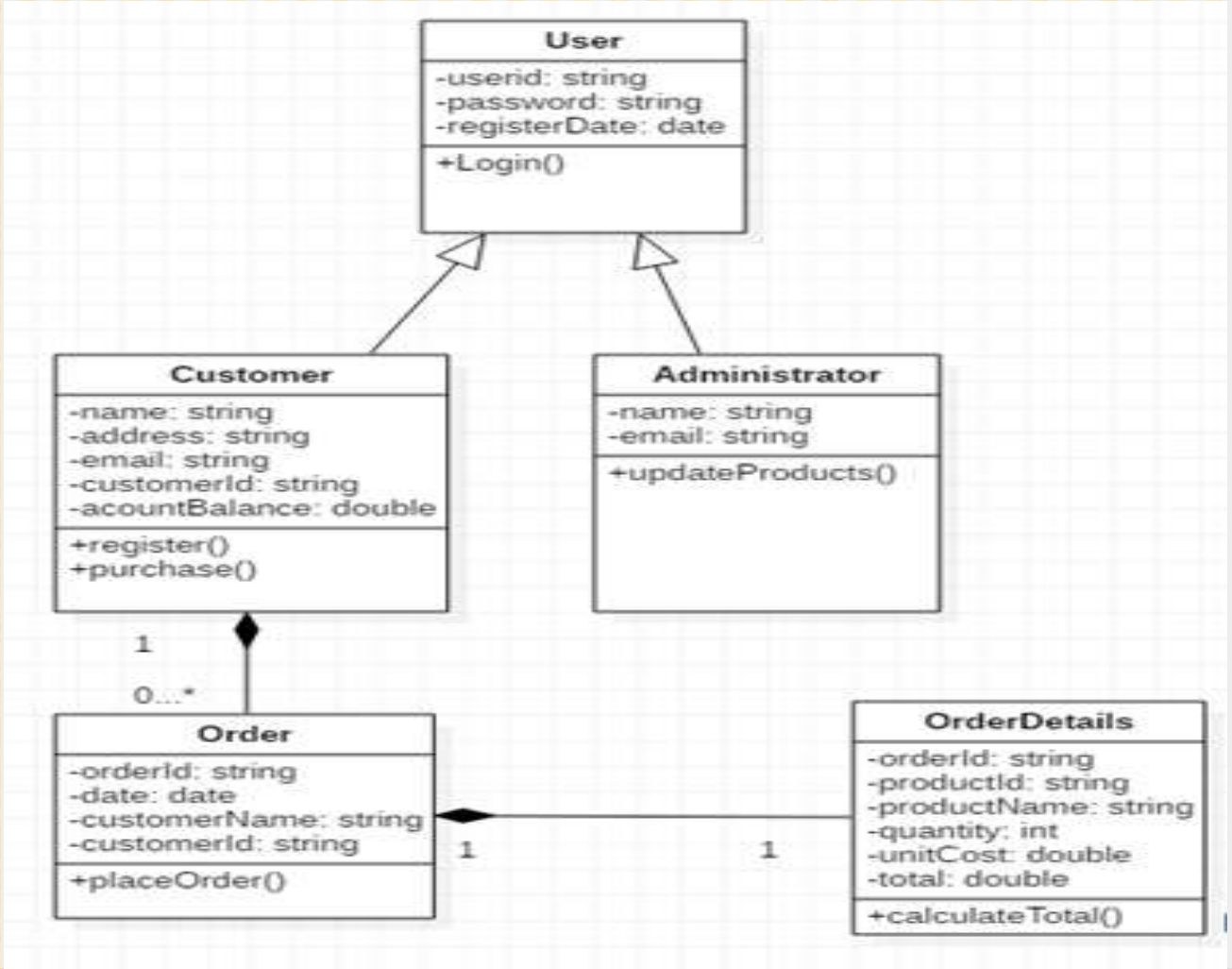
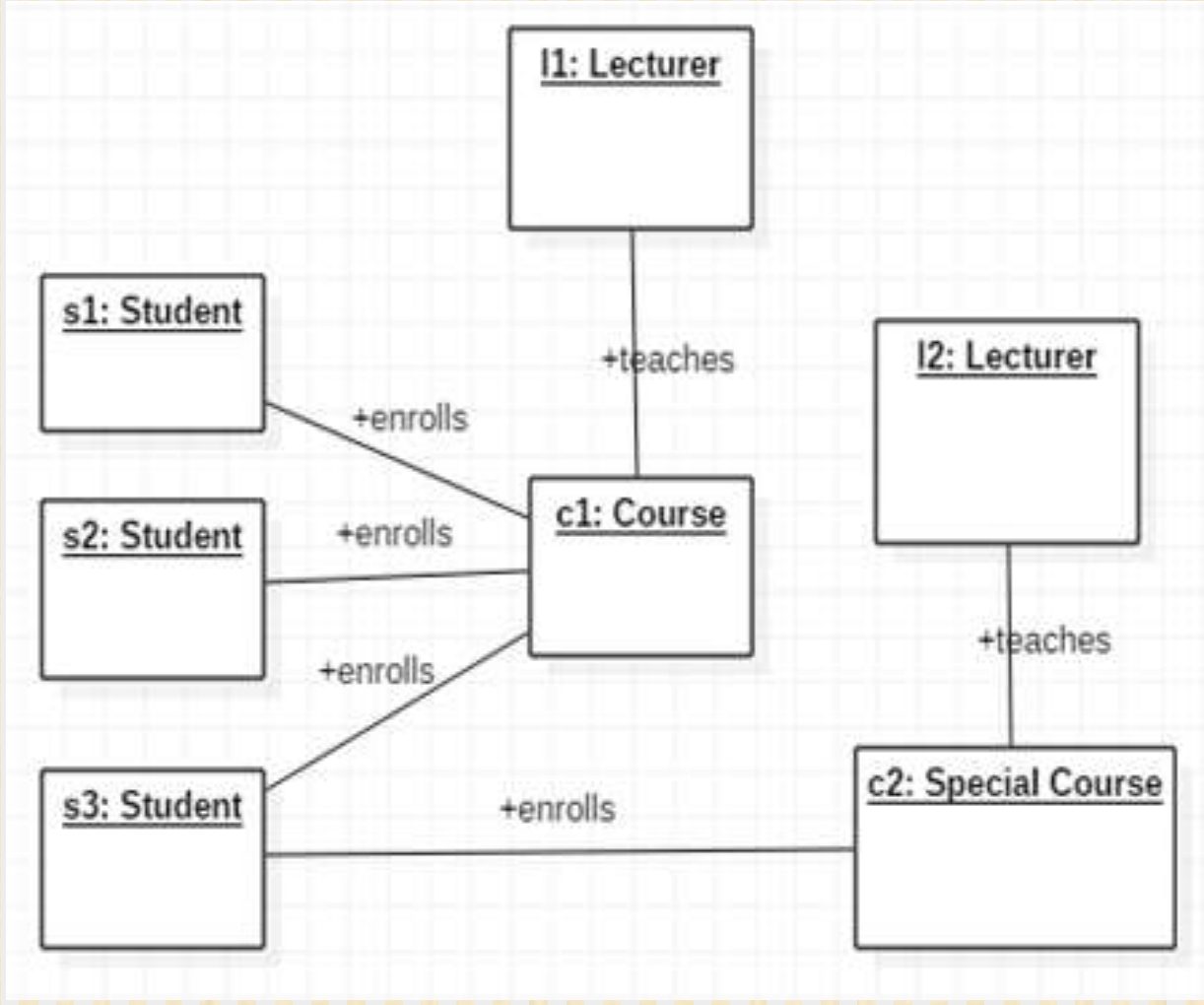
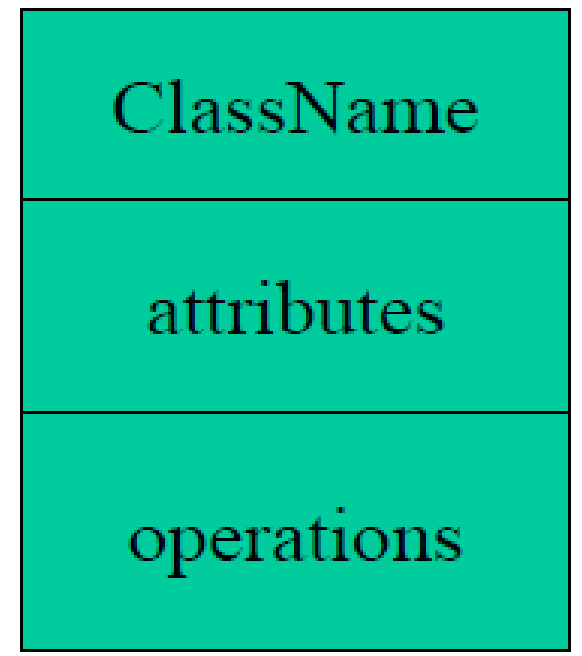


Diagram Types(Cont..)

- Another structural diagram is an object diagram. It is similar to a class diagram, but it focuses on objects.
- The basic concepts of **object diagram** are similar to a class diagram. These diagrams help to understand object behavior and their relationships at a particular moment.
- The **s1**, **s2**, and **s3** are student objects, and they enroll to c1 course object. The l1 lecturer object teaches the course **c1**. The lecturer object **l2** teaches the special course **c2**. The Student **s3** enrolls to c1 course as well as **c2** special course. This diagram illustrates how a set of objects relates to each other.

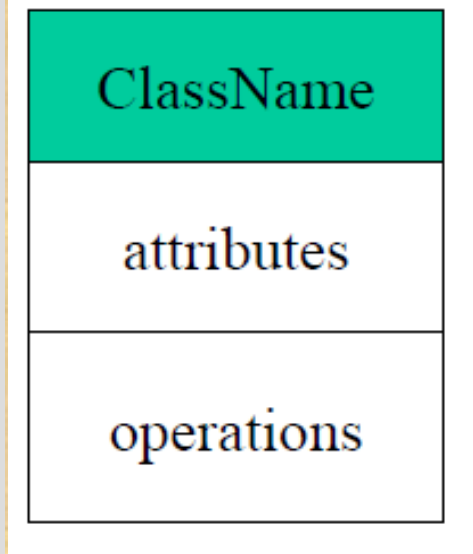


Essential elements Class Diagram



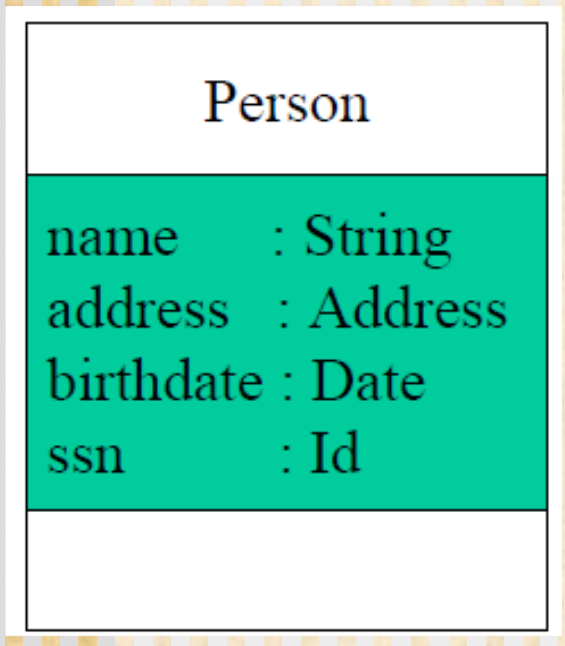
- Class Name
- Attributes
- Operations

Class Names



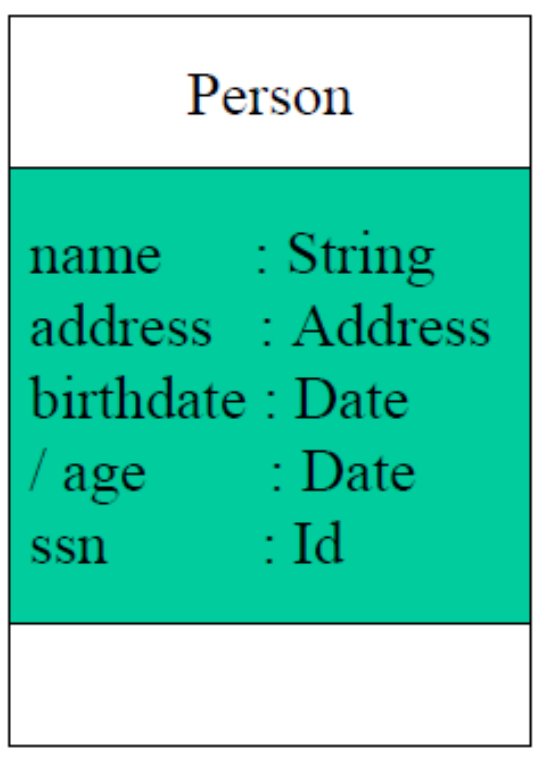
- The name of the class is the only required tag in the graphical representation of a class. It always appears in the top-most compartment.

Class Attributes



- An *attribute* is a named property of a class that describes the object being modeled.
- In the class diagram, attributes appear in the second compartment just below the name-compartment.

Class Attributes (Cont'd)



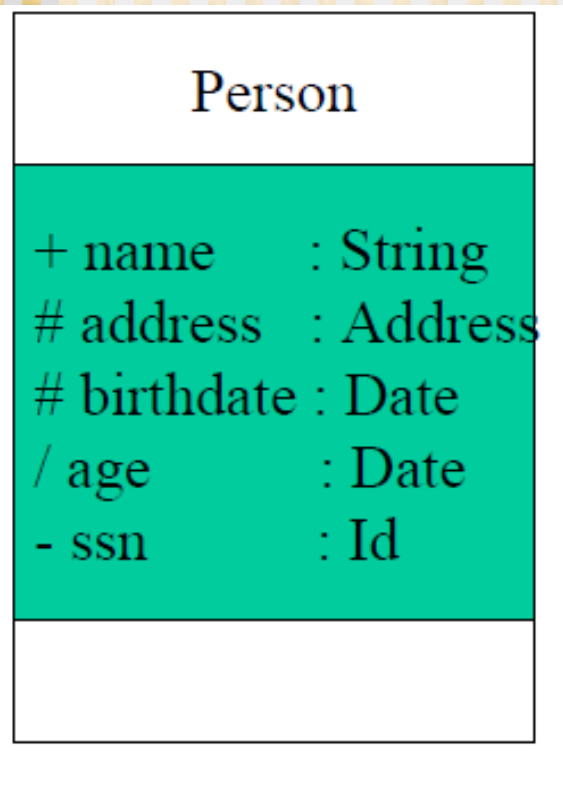
- Attributes are usually listed in the form:

  attributeName : Type

- A ***derived* attribute** is one that can be computed from other attributes, but doesn't actually exist. For example, a Person's age can be computed from his birth date. A derived attribute is designated by a preceding '/' as in:

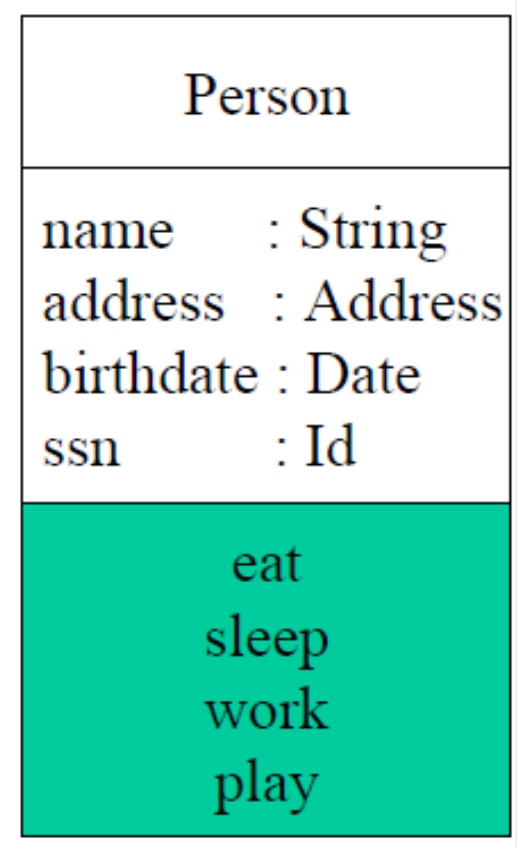
  / age : Date

Class Attributes (Cont'd)



- Attributes can be:
  - + public
  - # protected
  - - private
  - / derived

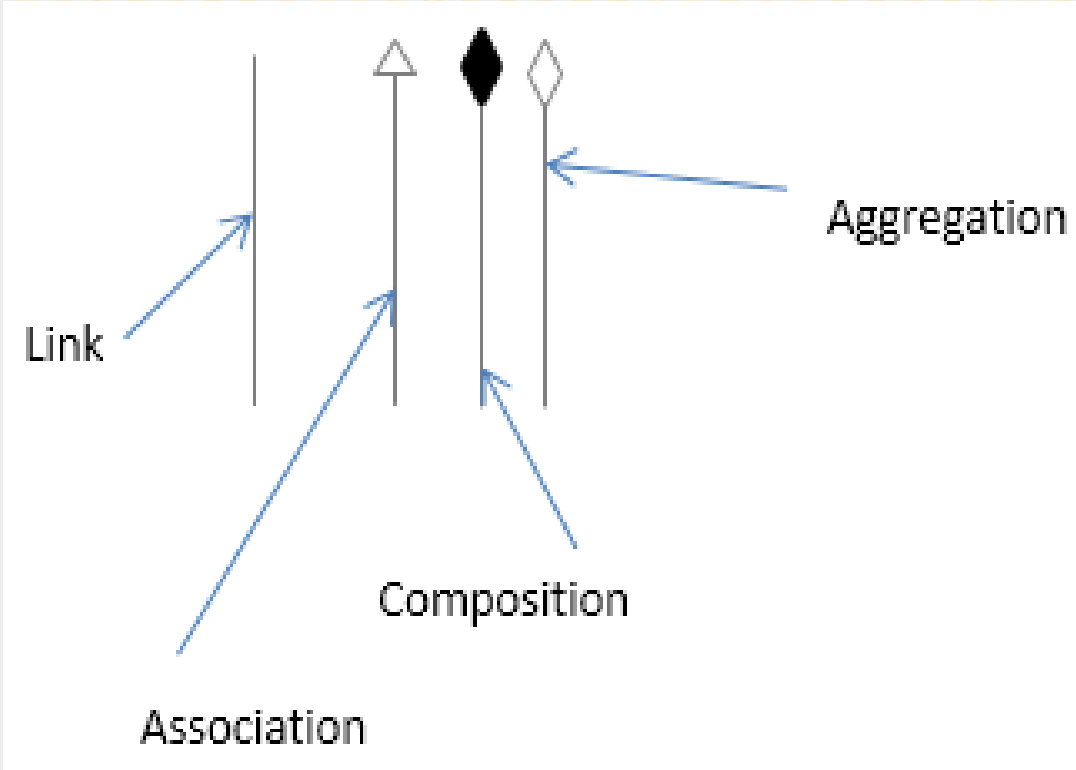
Class Operations



- *Operations* describe the class behavior and appear in the third compartment.

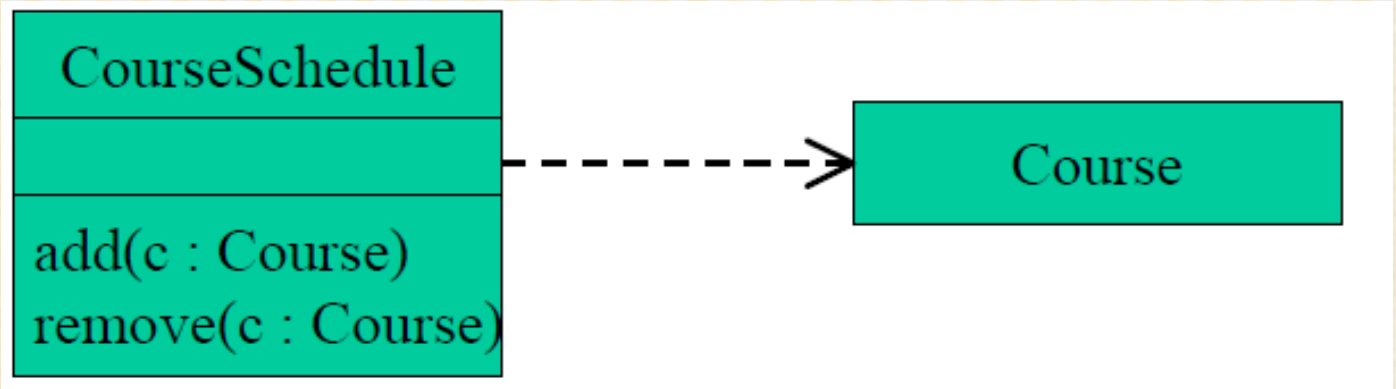
Relationships

- In UML, object interconnections (logical or physical), are modeled as relationships.
- There are three kinds of relationships in UML:
  - Dependencies
  - Generalizations
  - Associations

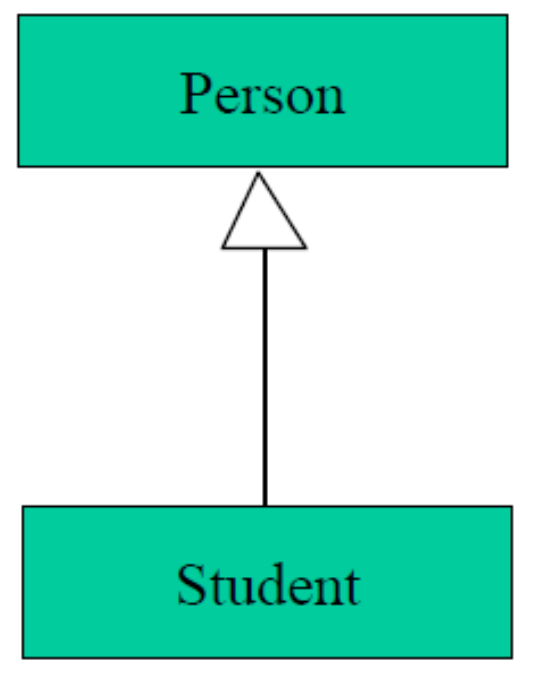


Dependency Relationships

- A *dependency* indicates a semantic/notational relationship between two or more elements.
- CourseSchedule has dependency on Course



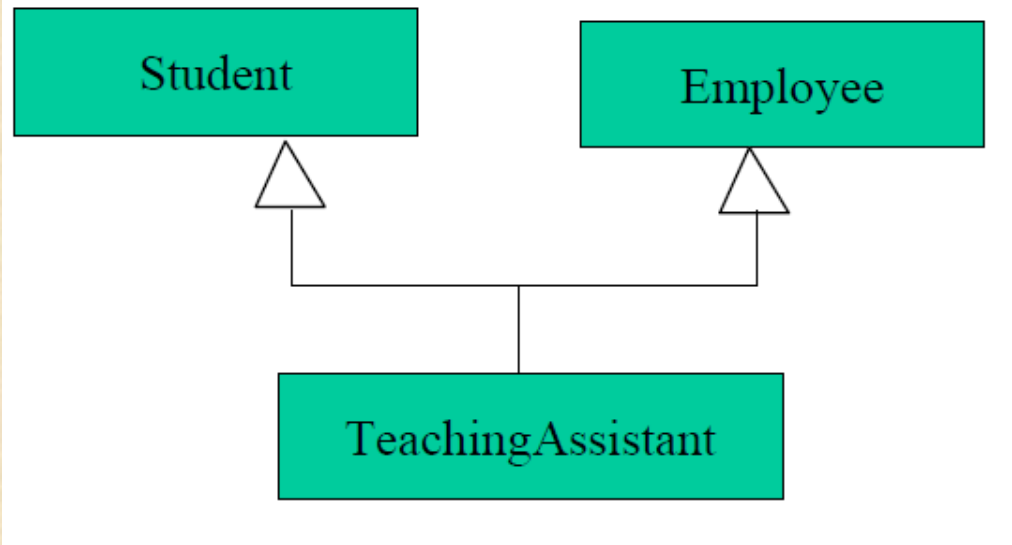
Generalization Relationships



- A *generalization* connects a subclass to its superclass.
- It denotes an inheritance of attributes and behavior from the superclass to the subclass and indicates a specialization in the subclass of the more general superclass.

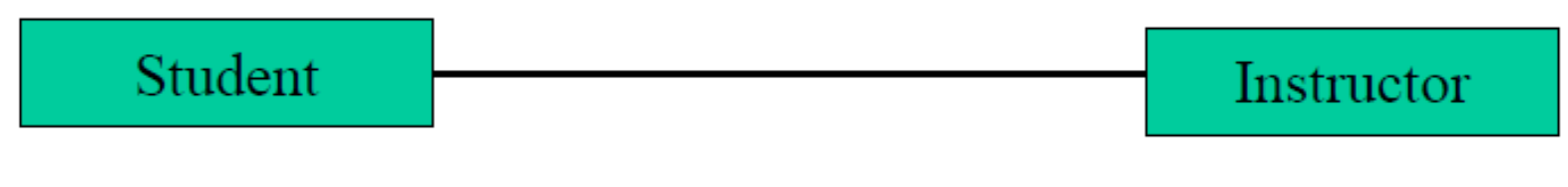
Generalization Relationships (Cont'd)

- UML permits a class to inherit from multiple super-classes, although some programming languages (*e.g.*, Java) do not permit multiple inheritance.



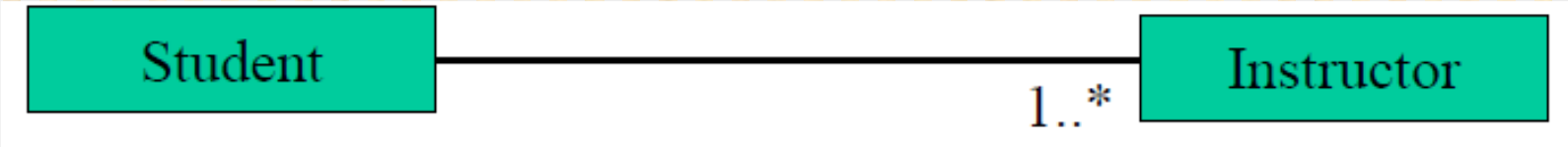
Association Relationships

- If two classes in a model need to communicate with each other, there must be link between them.
- An *association* denotes that link.



Association Relationships (Cont'd)

- We can indicate the *multiplicity* of an association by adding *multiplicity adornments* to the line denoting the association.
- The example indicates that a ***Student* has one or more *Instructors*:**



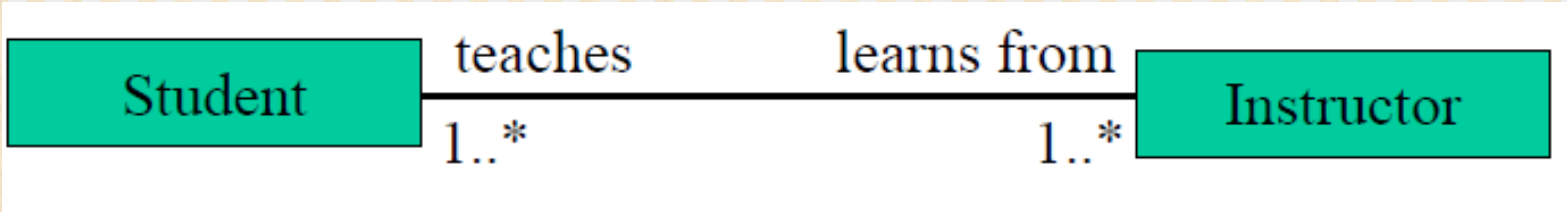
Association Relationships (Cont'd)

- The example indicates that **every *Instructor* has one or more**
- *Students*:

Association Relationships (Cont'd)

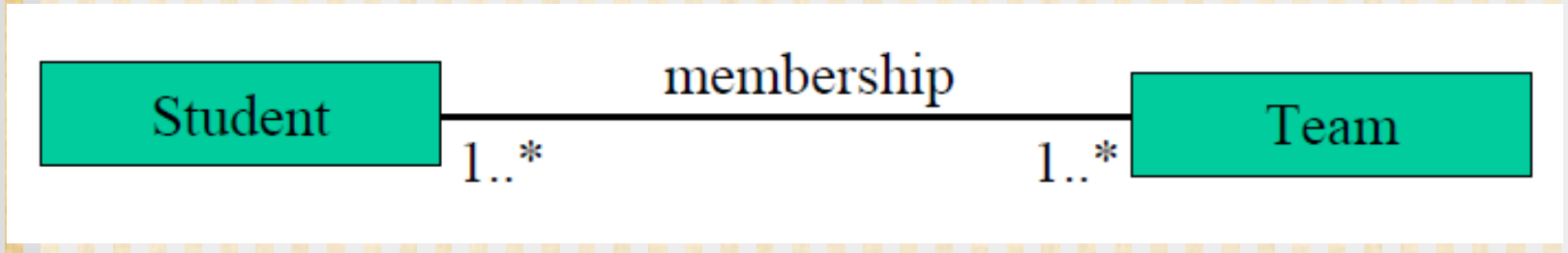
- We can also indicate the behavior of an object in an association

(*i.e.*, the *role* of an object) using *role names.*



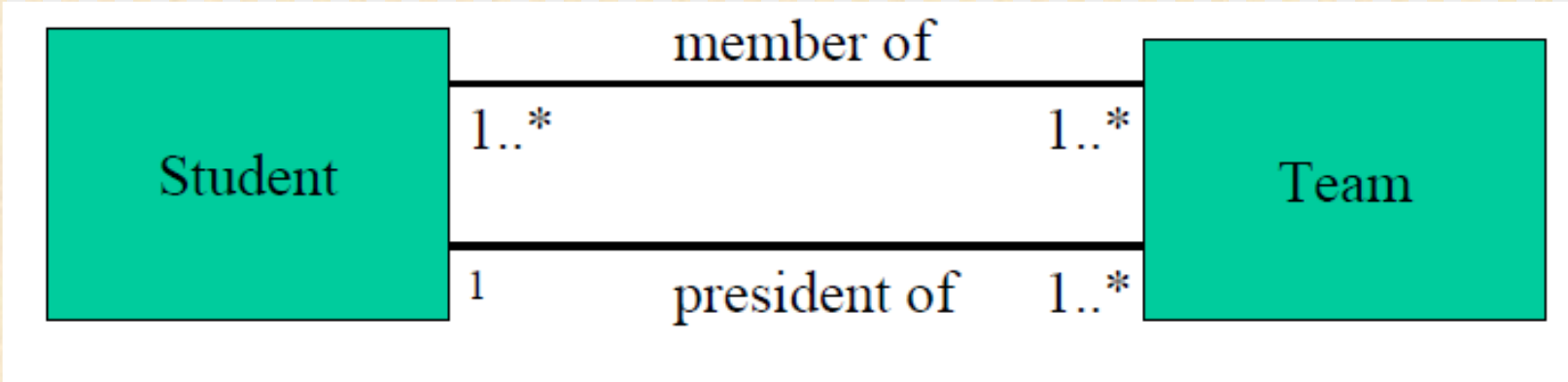
Association Relationships (Cont'd)

- We can also name the association.



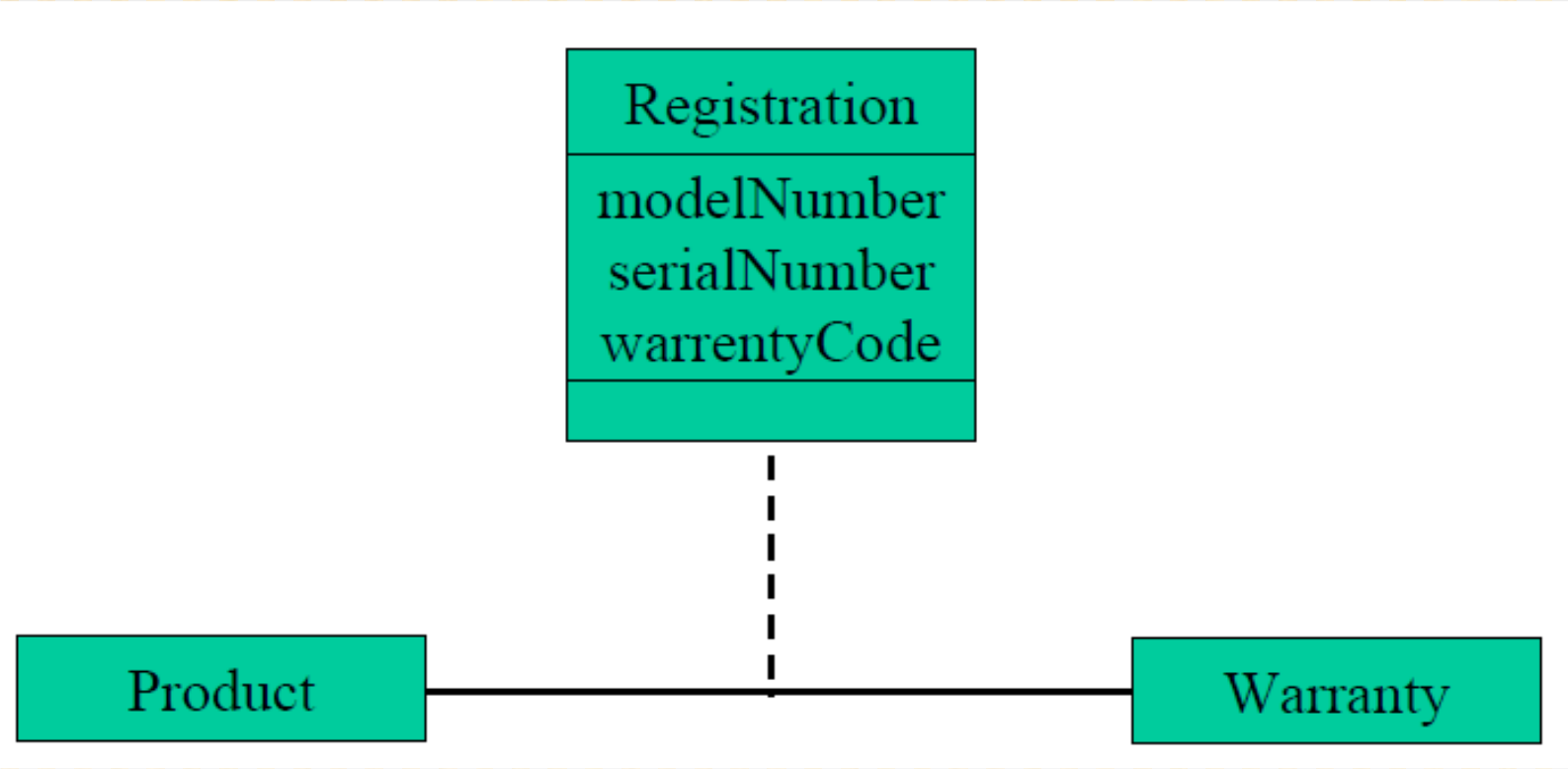
Association Relationships (Cont'd)

- We can specify dual associations.



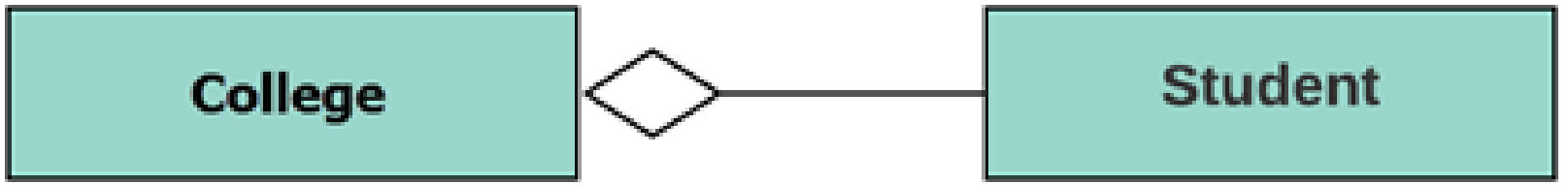
Association Relationships (Cont'd)

- Associations can also be objects themselves, called *link classes* or an *association classes*.



Relationships (Aggregation)

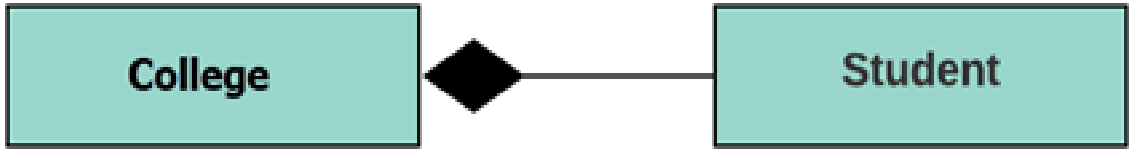
- Aggregation is a special type of association that models a whole- part relationship between aggregate and its parts.



- For example, the class college is made up of one or more student. In aggregation, the contained classes are never totally dependent on the lifecycle of the container.
- Here, the college class will remain even if the student is not available.

Relationships (Composition)

- Compositions are denoted by a filled-diamond adornment on the association.



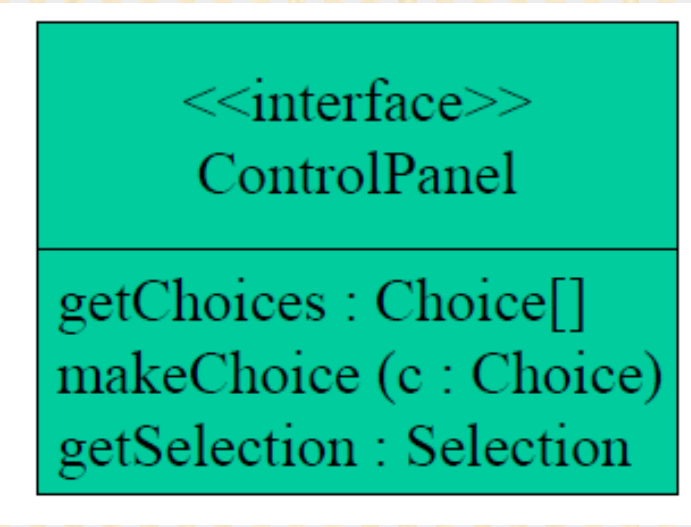
- The composition is a special type of aggregation which denotes strong ownership between two classes when one class is a part of another class.
- For example, if college is composed of classes student. The college could contain many students, while each student belongs to only one college. So, if **college is not functioning all the students also removed.**

Interfaces

- An *interface* is a named set of operations that specifies the behavior of objects without showing their inner structure.
- It can be rendered in the model by a one- or two-compartment rectangle, with the *stereotype* <<interface>> above the interface name.

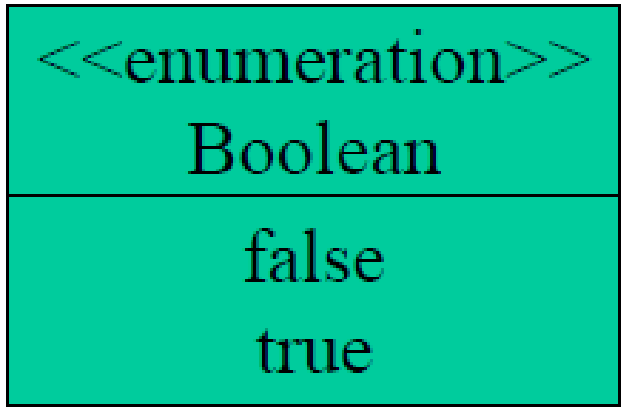
Interface Services

- Interfaces do not get instantiated. They have no attributes or state. Rather, they specify the services offered by a related class.

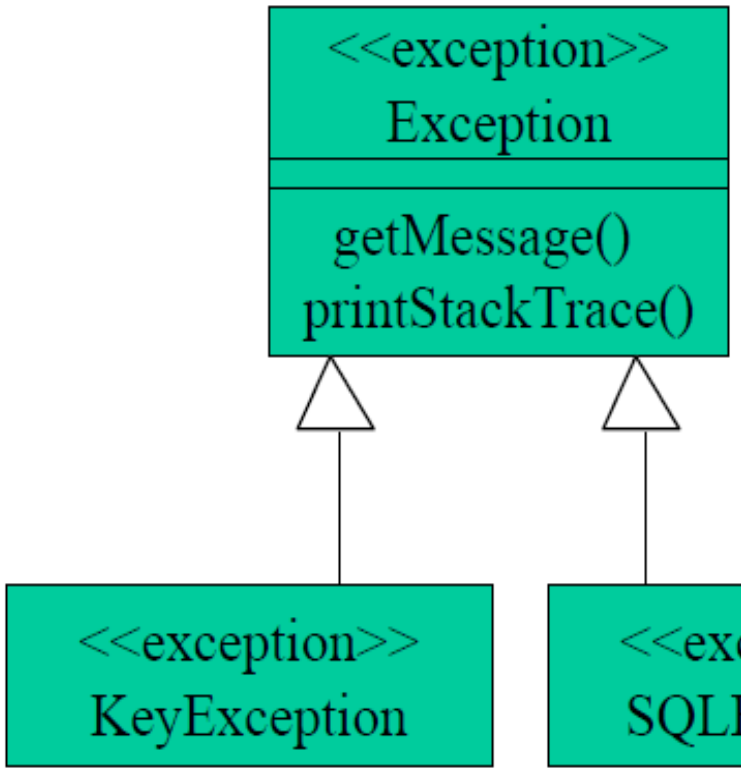


Enumeration

- An *enumeration* is a user-defined data type that consists of a name and an ordered list of enumeration literals.



Exceptions

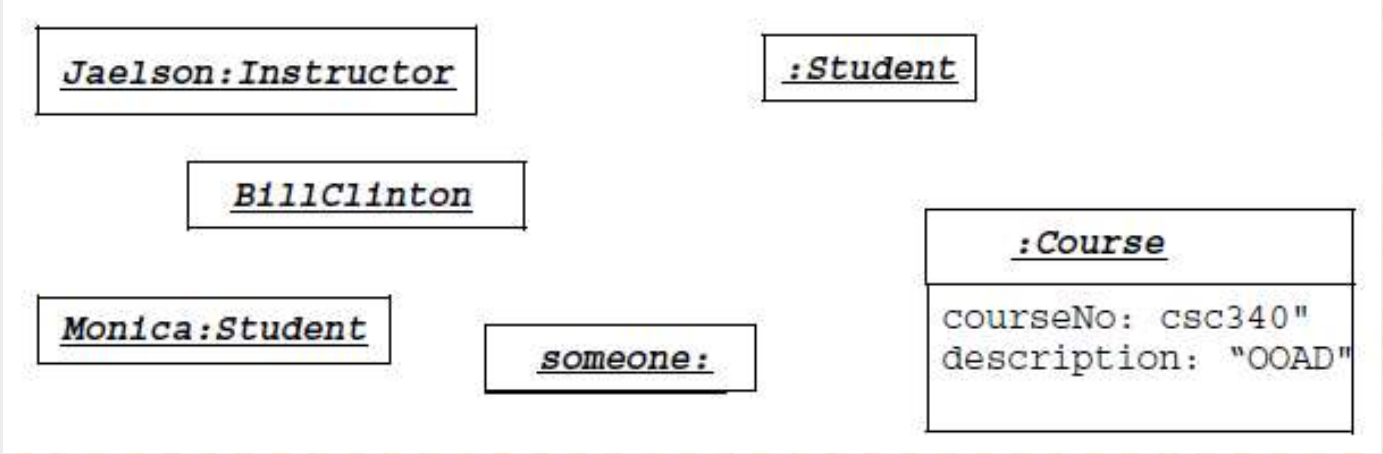


Exceptions can be modeled just like any other class.

Notice the <<exception>> stereotype in the name compartment.

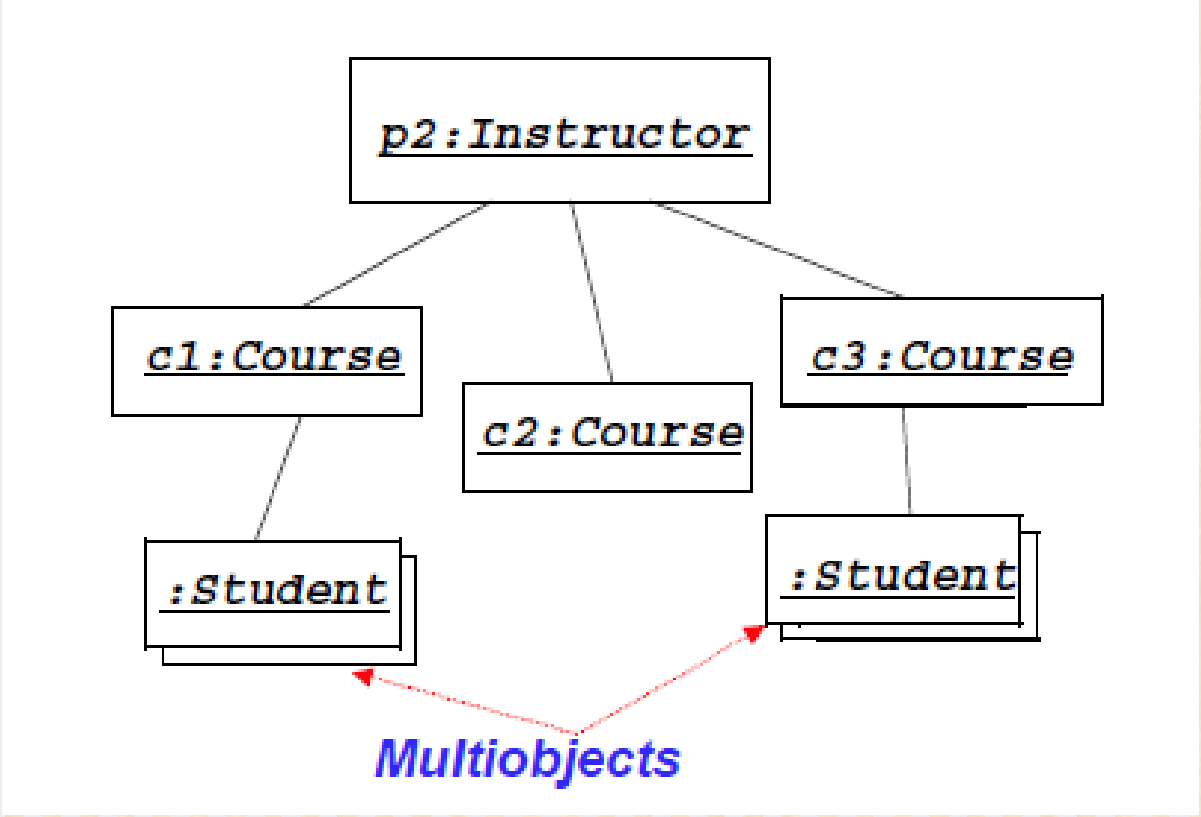
Object Diagrams

- *Model the instances of things described by a class.*
- *Each object diagram shows a set of objects and their interrelationships at a point in time.*
- *Used to model a snapshot of the application.*
- *Each object has an optional name and set of classes it is an instance of, also values for attributes of these classes.*

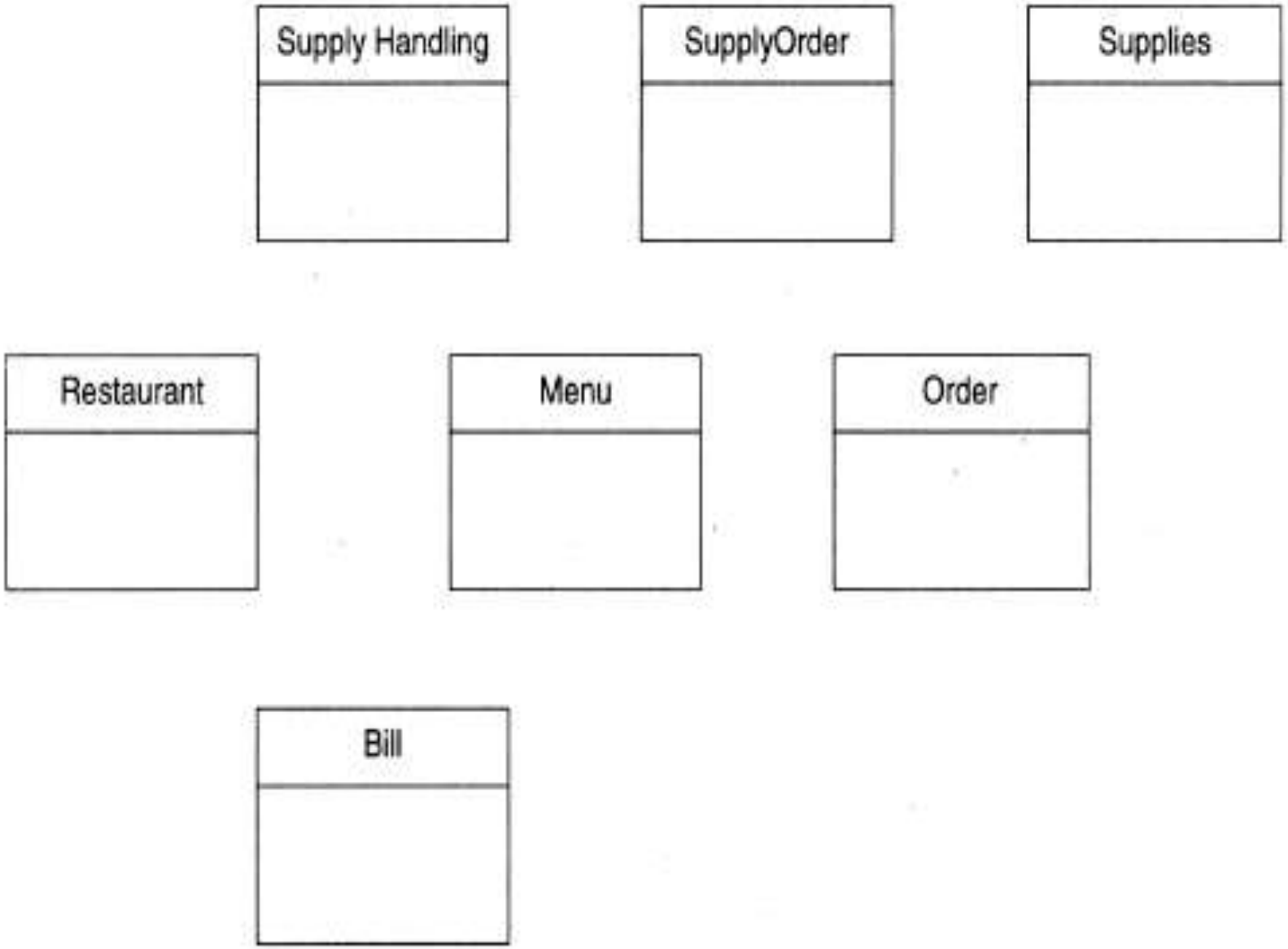


Multi objects

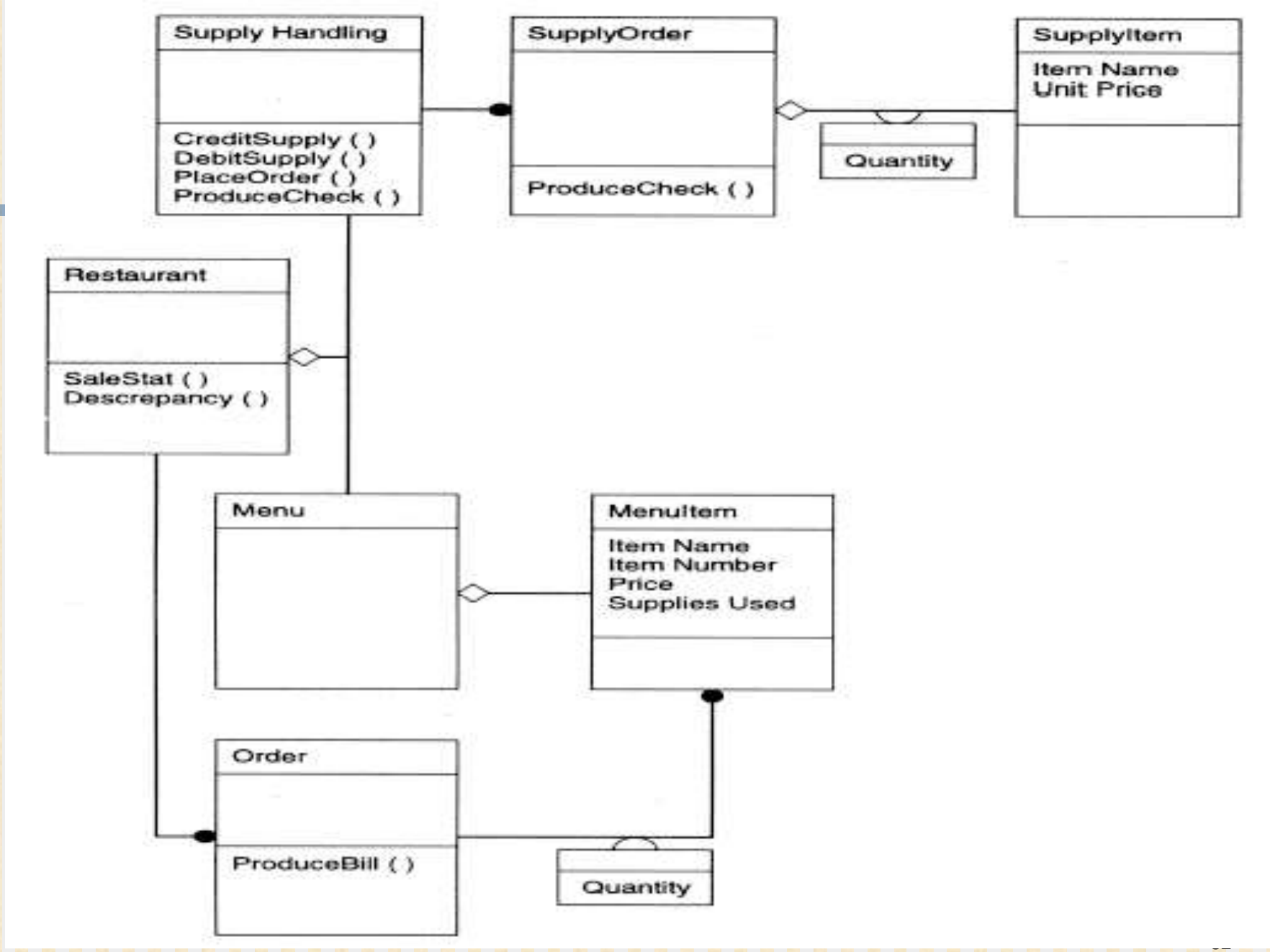
- *A **multi object** is a set of objects, with an undefined number of elements*



Restaurant example: Initial classes

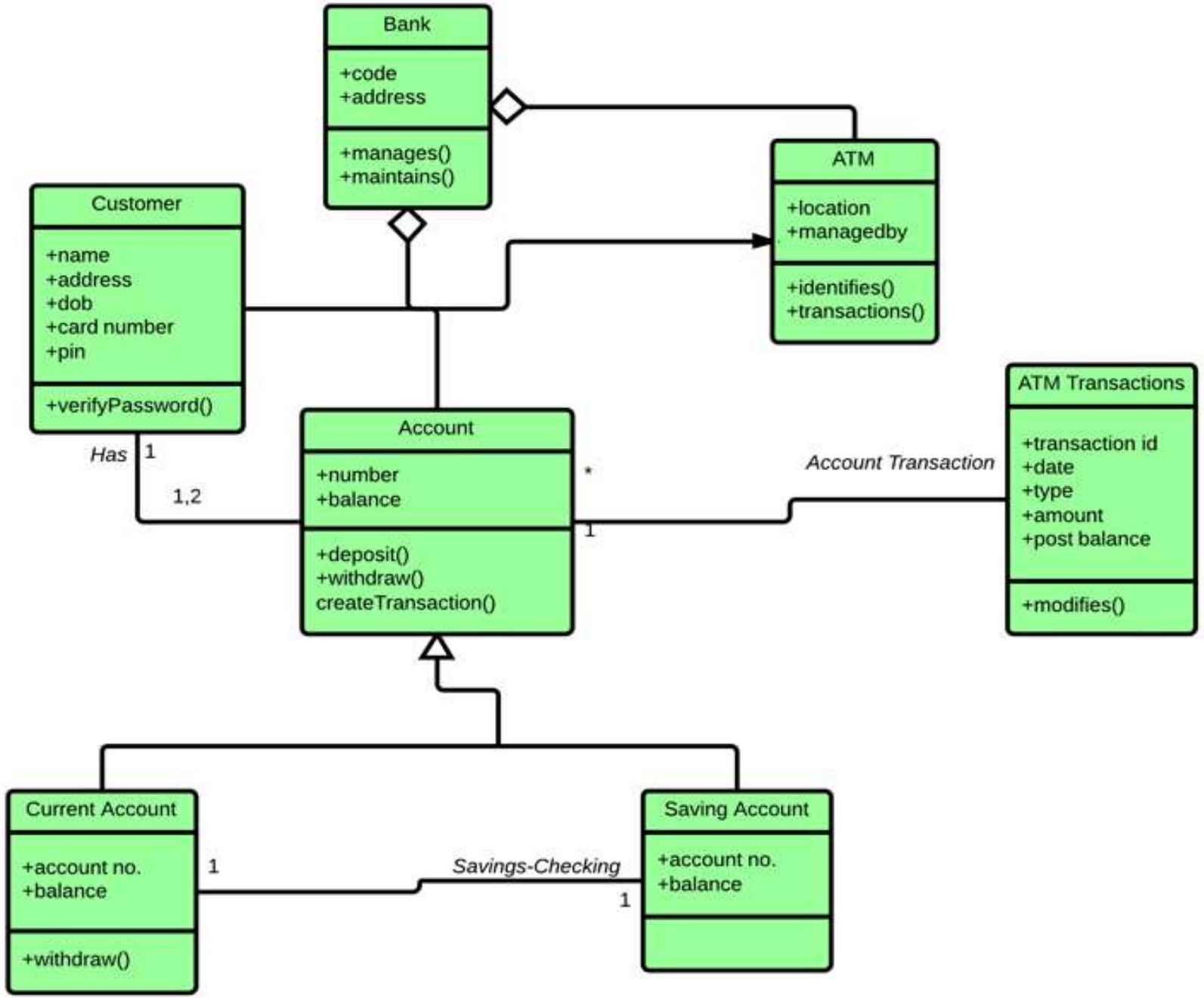


Restaurant example: Initial classes

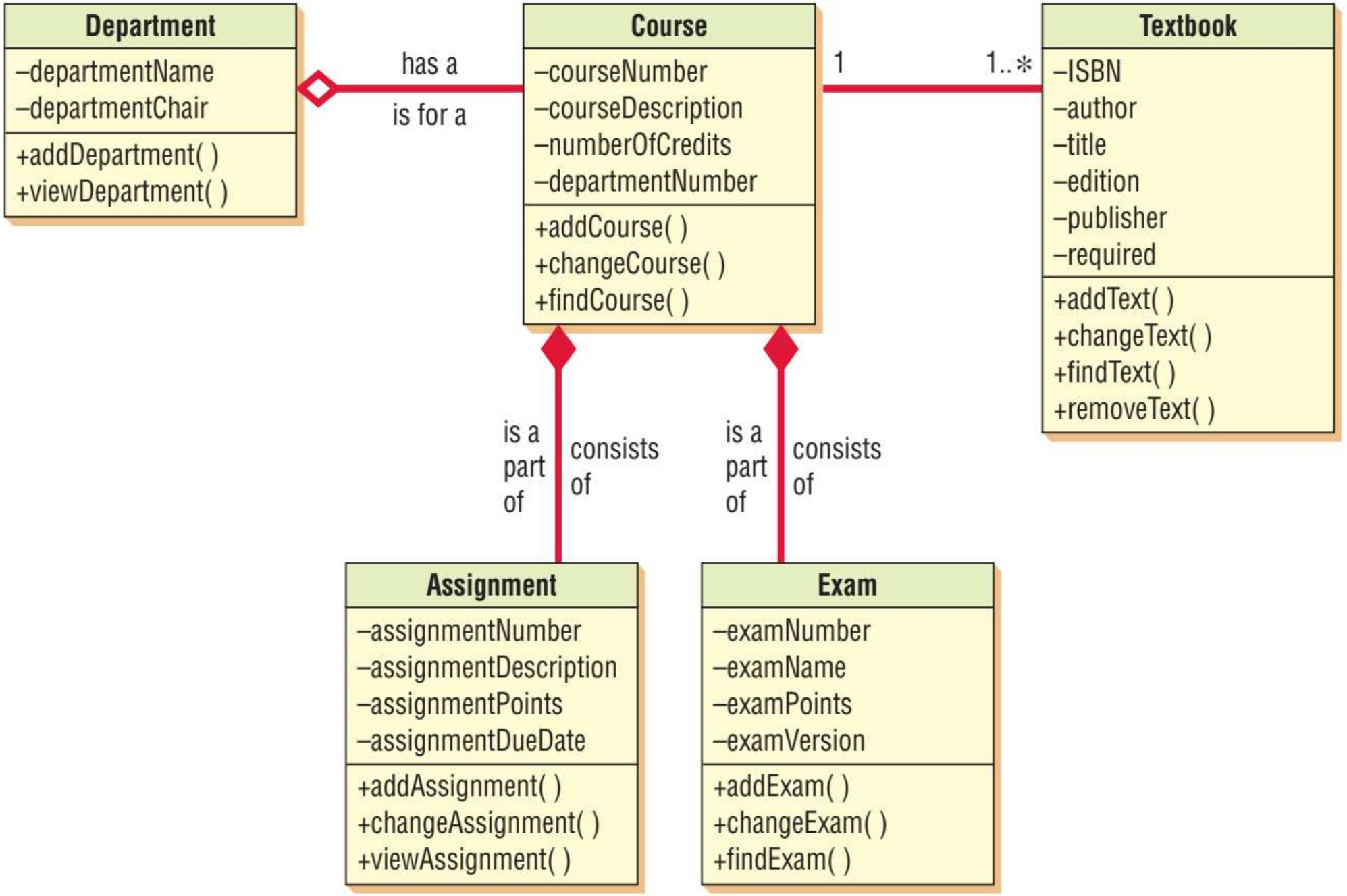


Example of UML Class Diagram

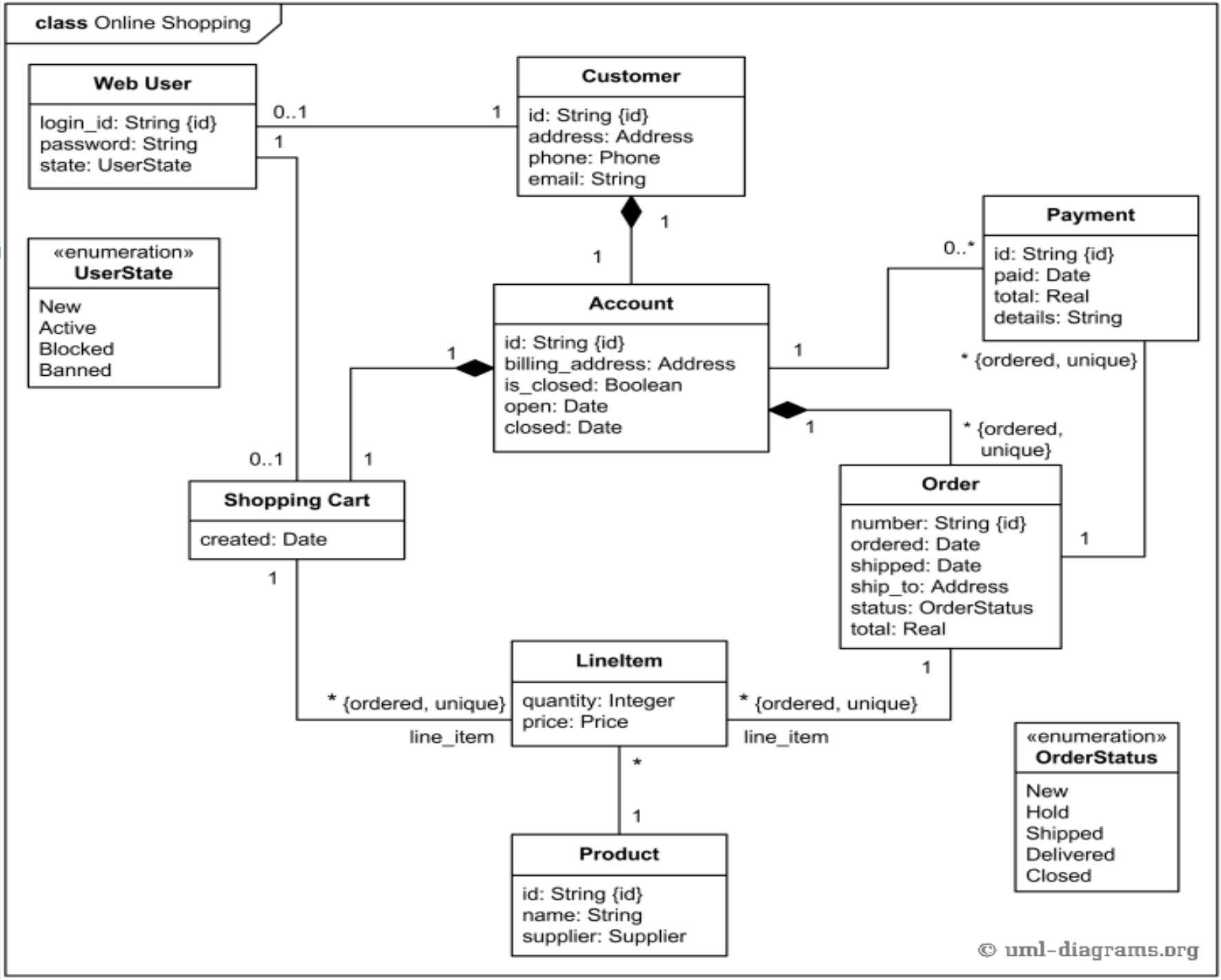
ATMs system is very simple as customers need to press some buttons to receive cash. However, there are multiple security layers that any ATM system needs to pass. This helps to prevent fraud and provide cash or need details to banking customers



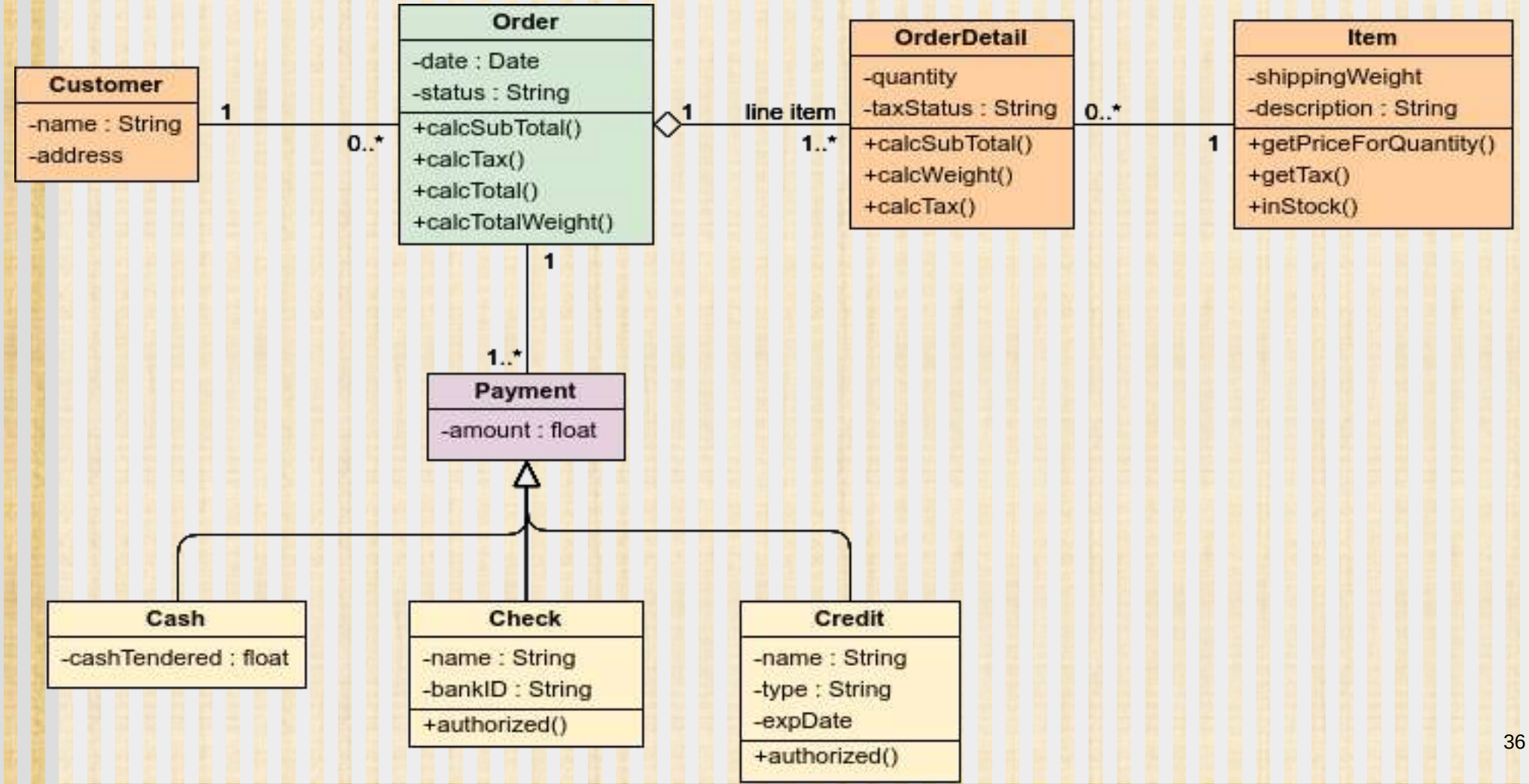
A class diagram for course offerings



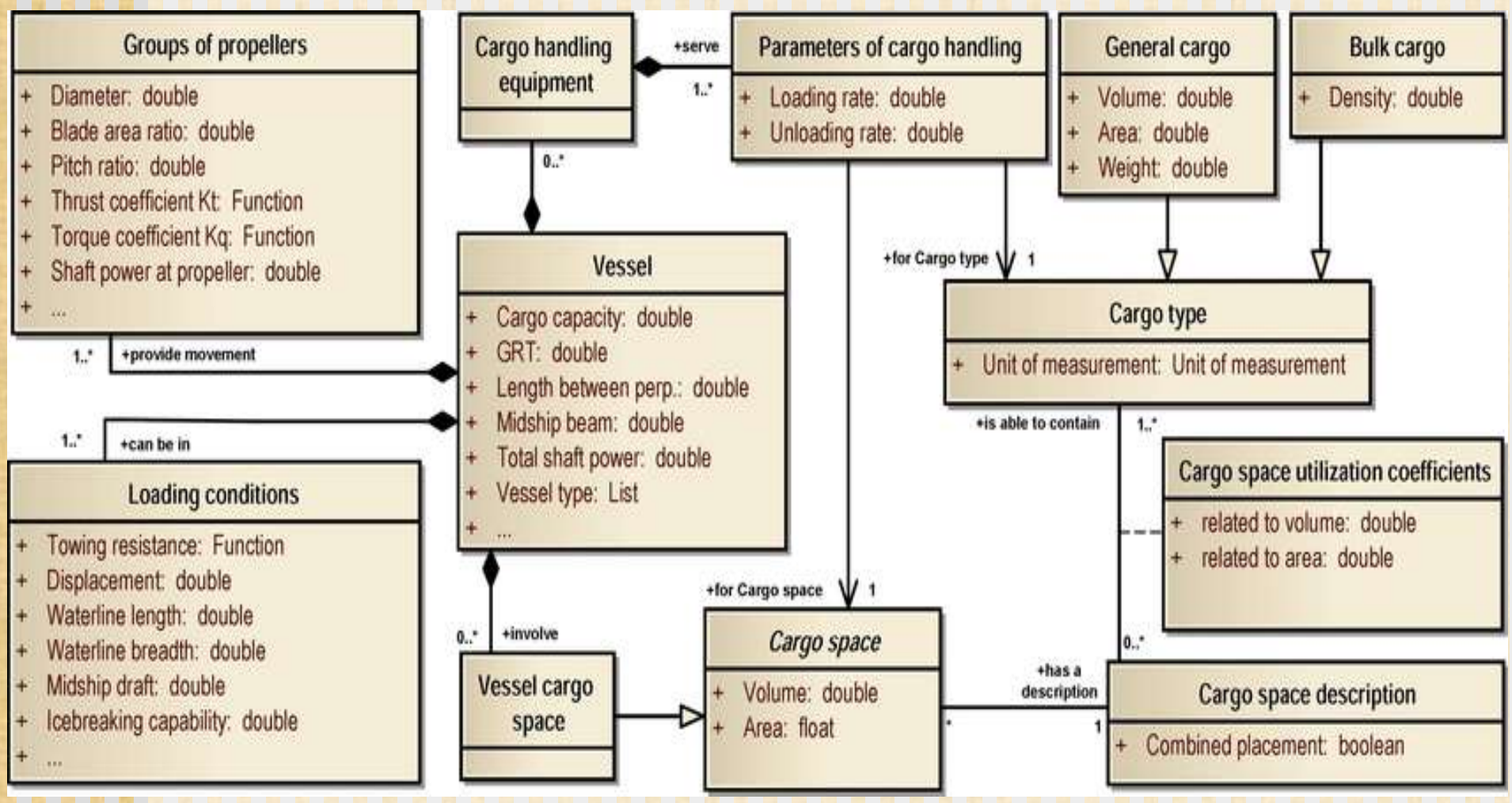
Class Diagram: Online Shopping



Class Diagram: Order Process



Ship & Cargo Object Model



References

1. **Software Engineering A practitioner's Approach** by Roger S. Pressman, 7th edition, McGraw Hill, 2010.
2. **Software Engineering by Ian Sommerville,** 9th edition, Addison-Wesley, 2011
3. **Systems Analysis and Design,** Kendall and Kendall, Fifth Edition

8. Business Process Modeling

CSE333: Software Engineering

Abdus Sattar
Assistant Professor
Department of Computer Science and Engineering
Daffodil International University
Email: abdus.cse@diu.edu.bd

Discussion Topics

- Business process modeling(BMP)
- Notation defining workflows
- Some rules for creating BPN
- BPM example and practicing
- Why does BPM uses?
- Integrating Requirements and Business Process Models in BPM Projects(Online Materials)

Business Process Model(BPM)

- Business process modeling is the graphical representation of a company's business processes or workflows with a visual guide, as a means of identifying potential improvements.
- Business process modelling represents:
  - Business activities,
  - Information flow and,
  - Decision logic in business processes.

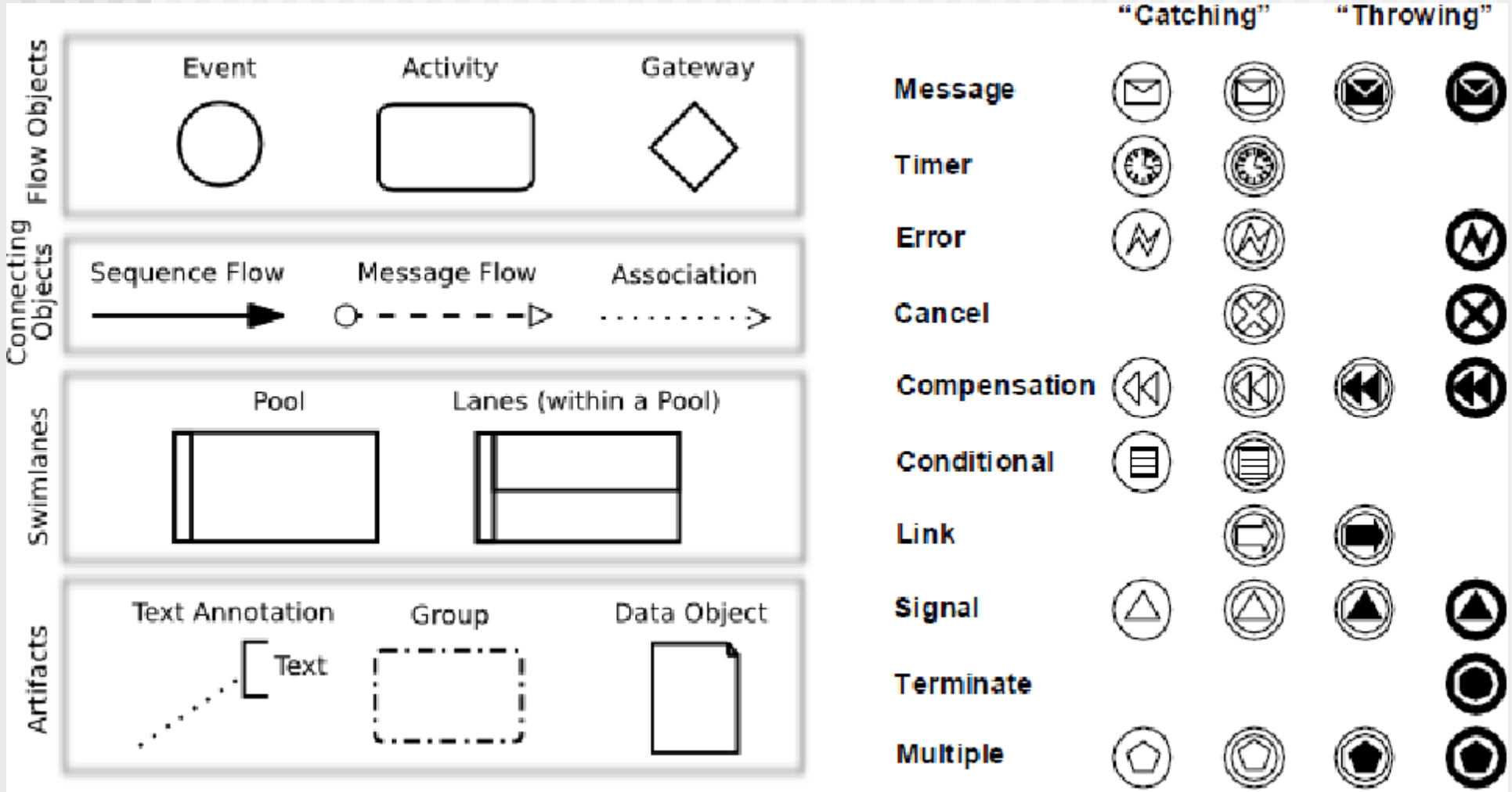
10 Business Process Modelling Techniques

- 1. Business process modelling notation (BPMN)
- 2. Unified Modelling language diagrams
- 3. Flow chart technique
- 4. Data flow diagrams
- 5. Role Activity diagrams
- 6. Role interaction diagrams(Sequence Diagram)
- 7. Gantt charts
- 8. Integrated definition for function modelling
- 9. Colored Petri nets
- 10. Object oriented methods

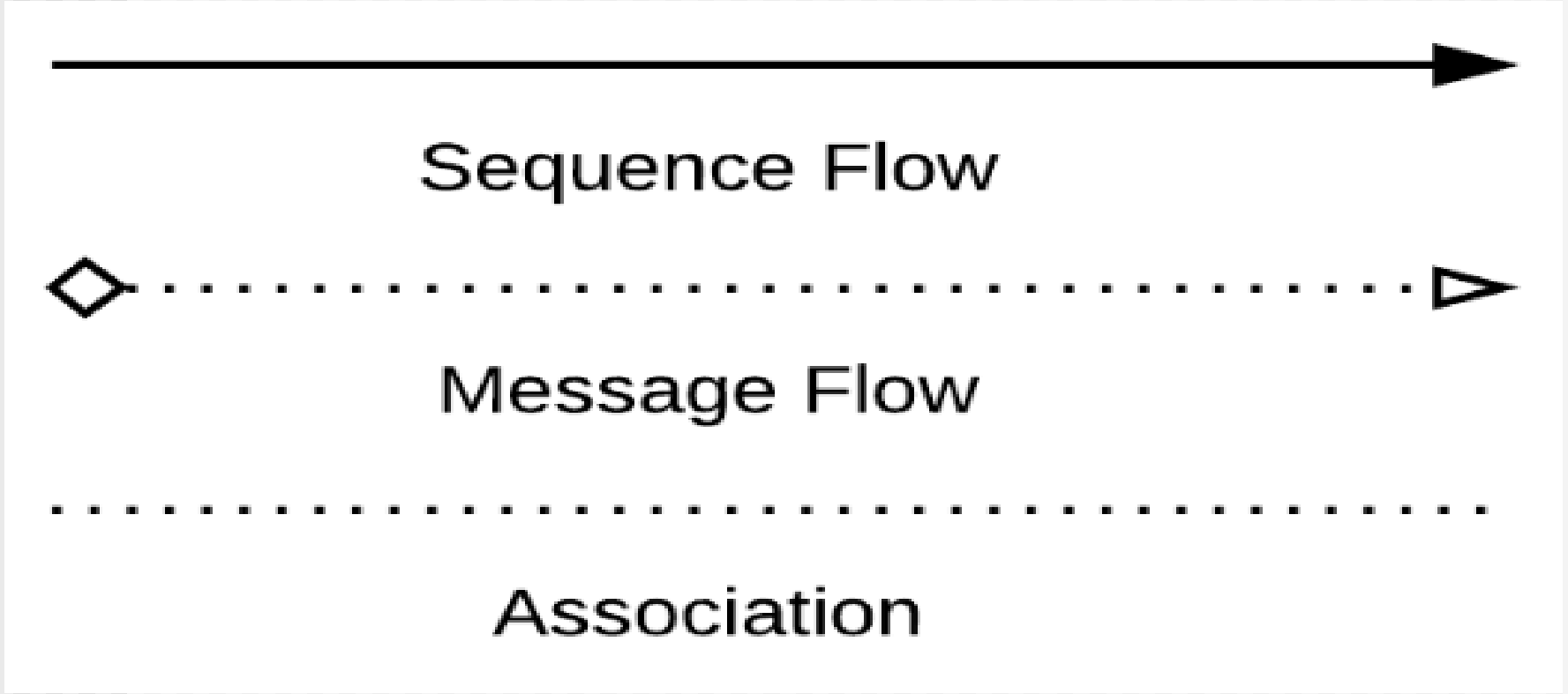
Elements of Business process modelling notation (BPMN)

- Business process modelling notation (BPMN) is comprised of symbols that are used as a representation of tasks and workflows.
- Any symbols can be used in your business process, but using standardized ones allows you to collaborate with outside analysts more easily, and it spares you from having to invent your own visual language.
- BPMN symbols fall into the following categories:
  - **Flow objects**. Shows the flow of the process and are represented as follows:
    - **Circles.** Events are displayed inside of circular shapes
    - **Rectangles.** Activities fit into rectangular boxes
    - **Diamonds.** Control points or gateways are represented as diamond shapes
  - **Connecting objects**. Used to show how tasks are connected, and in what sequence they occur
    - **Solid lines.** Shows task transfers
    - **Dashed lines.** Shows messages
  - **Swim lanes.** These make provision for sub processes that share responsibilities and how they interact. The swim lanes are the people or departments that the sub process impacts on

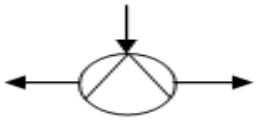
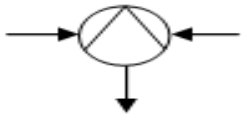
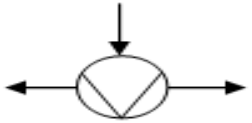
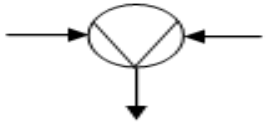
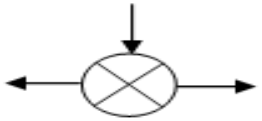
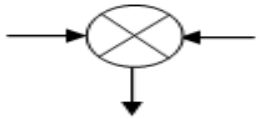
Elements of BPMN



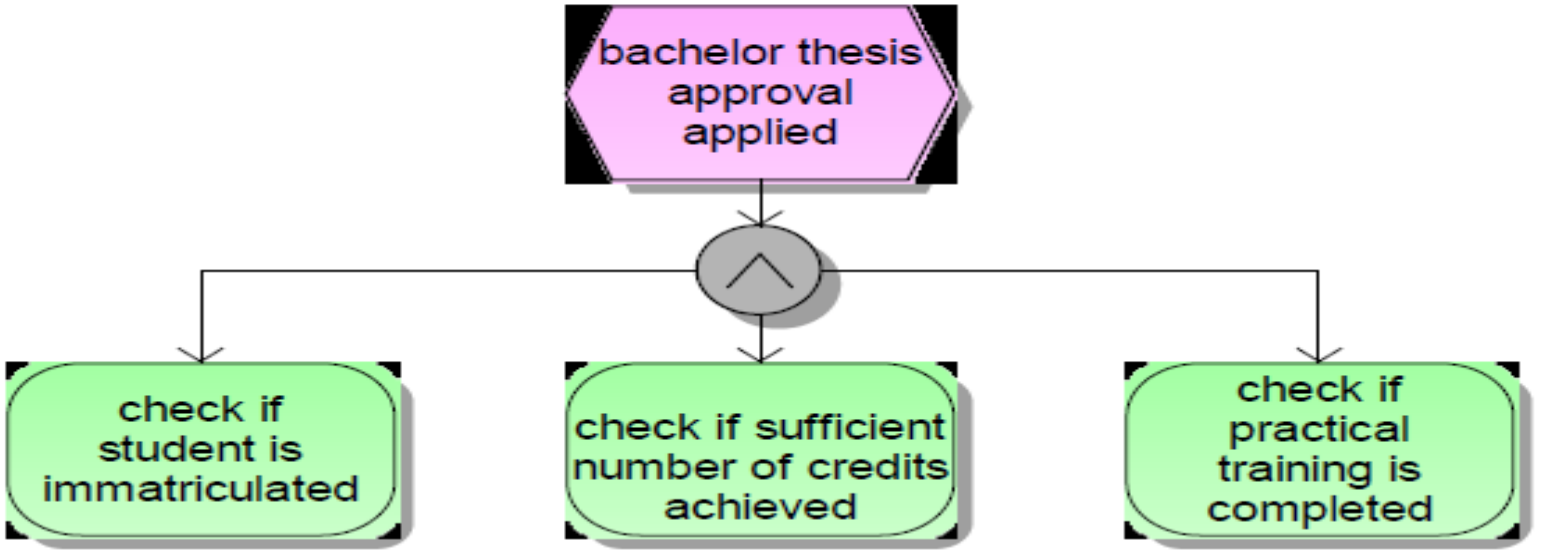
Elements of BPMN



Gateway Types

Konnec-tors:	AND	Split	
		Join	
	OR	Split	
		Join	
	XOR	Split	
		Join	

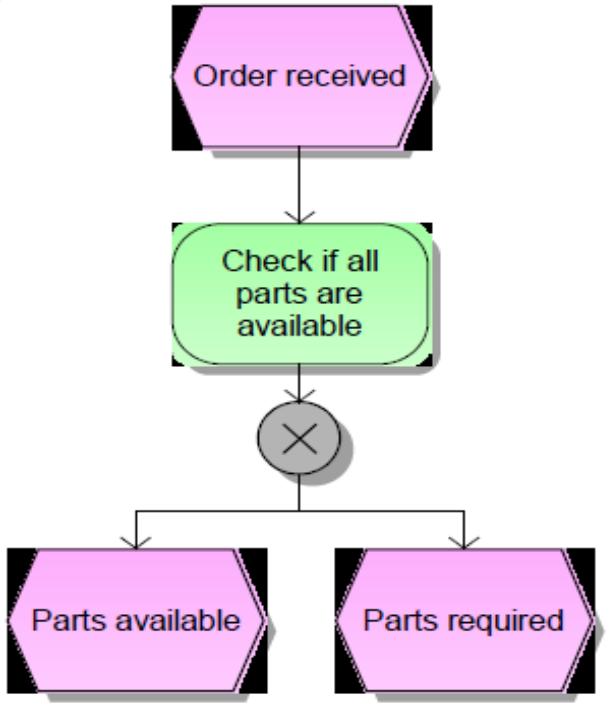
Logical Connectors

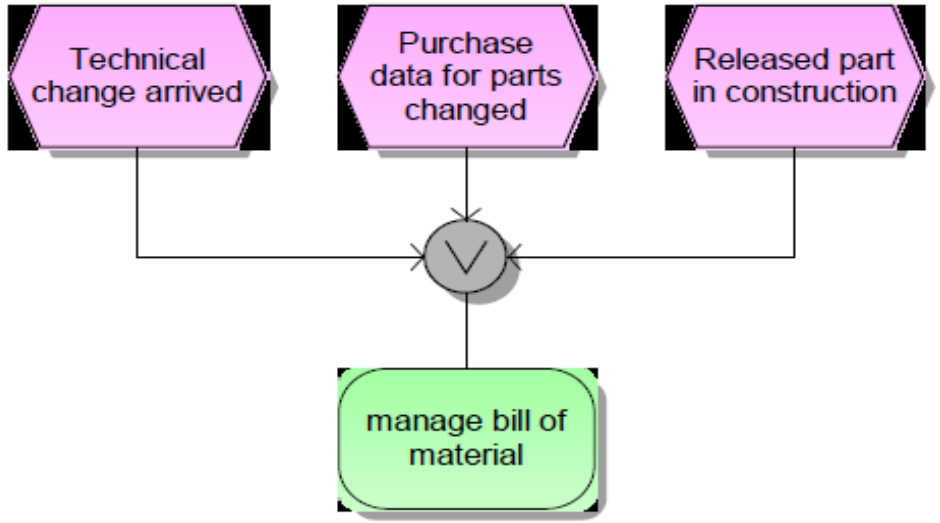


AND - Function

After the event, all the functions are started

Different Types of "OR"





Exclusive OR - Function

After the Function just one event is occuring

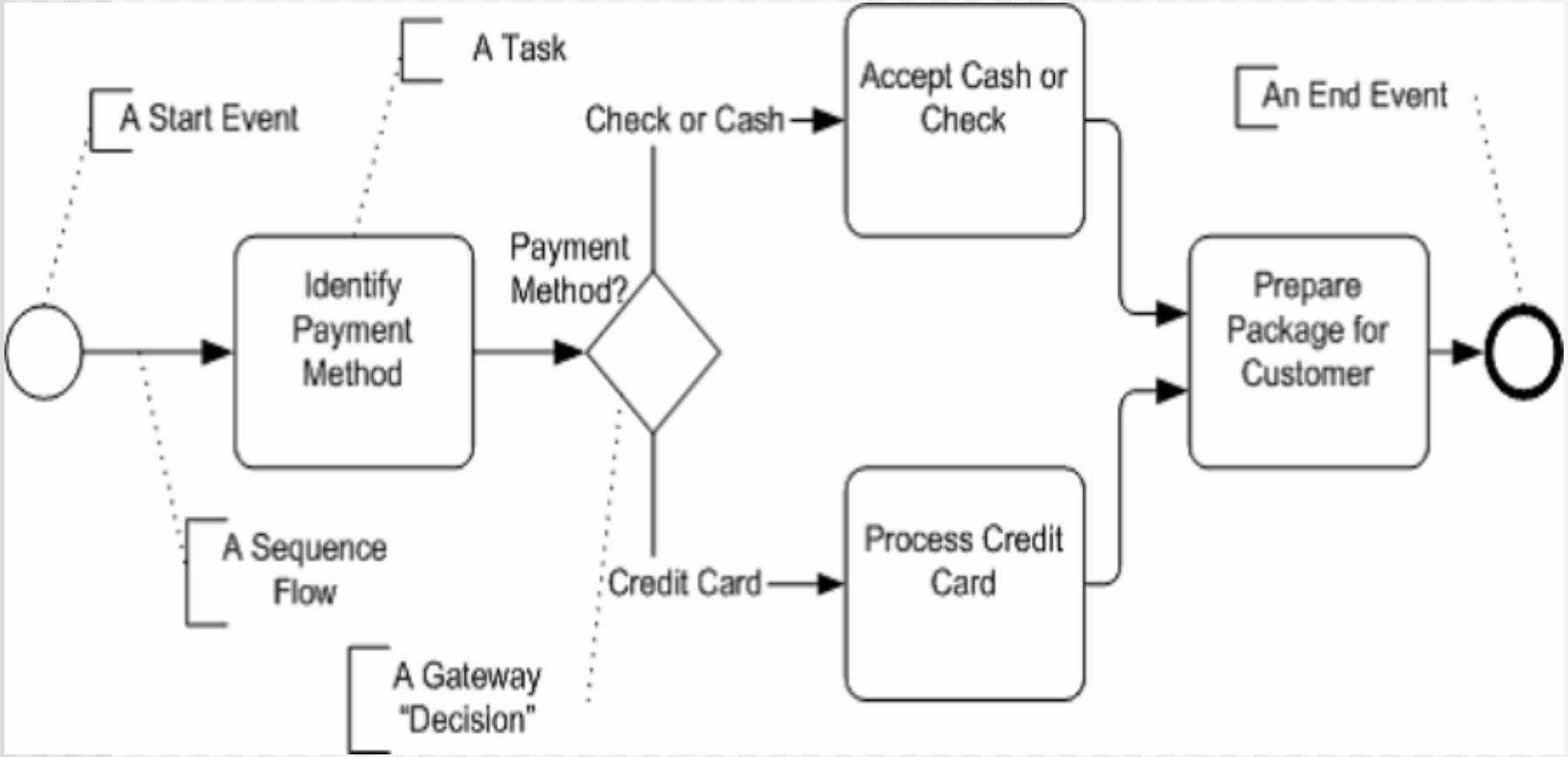
OR - Function

Function can only be started if one or more events have occurred

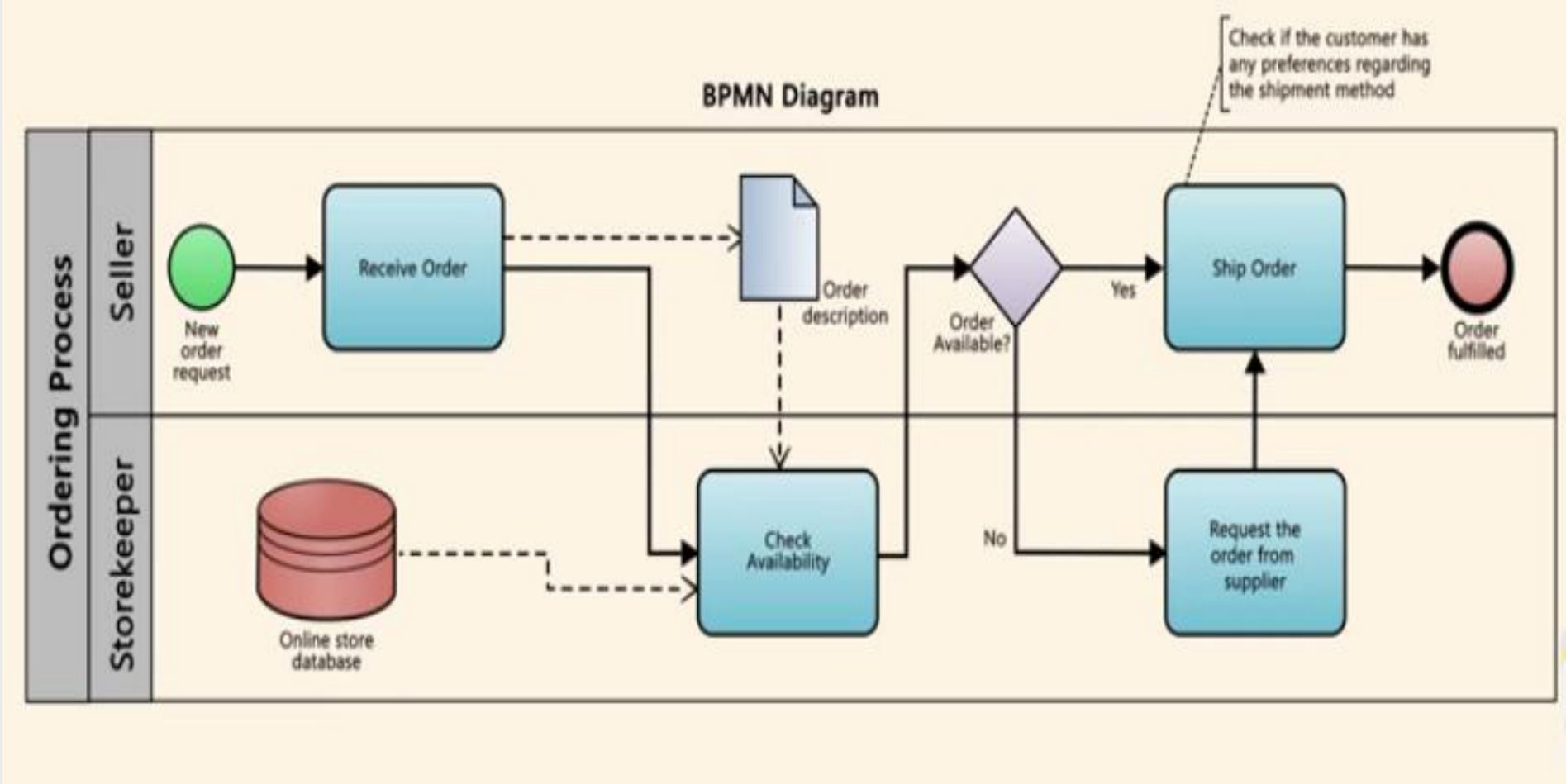
Some rules for BPMNs

- at least one function
- at least one start event
- at least one end event
- each function is preceded and followed by an event
- an event is preceded and followed by a function, except the start and end events
- each event and function has, at most, one ingoing and one outgoing flow
- splits and joins are realised via connectors
- an OR or XOR split cannot be invoked by an event (events cannot decide anything)

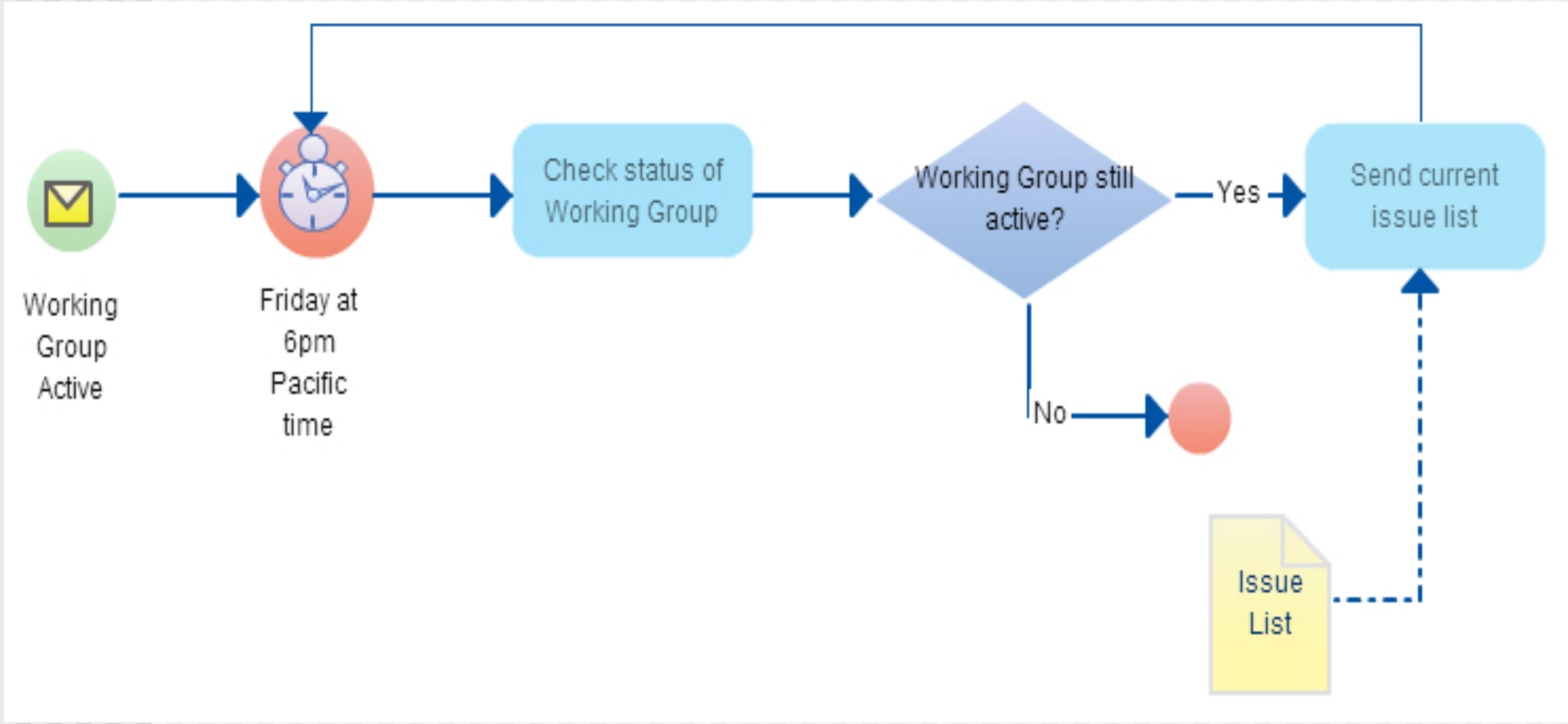
Business Process Model for Payment Process



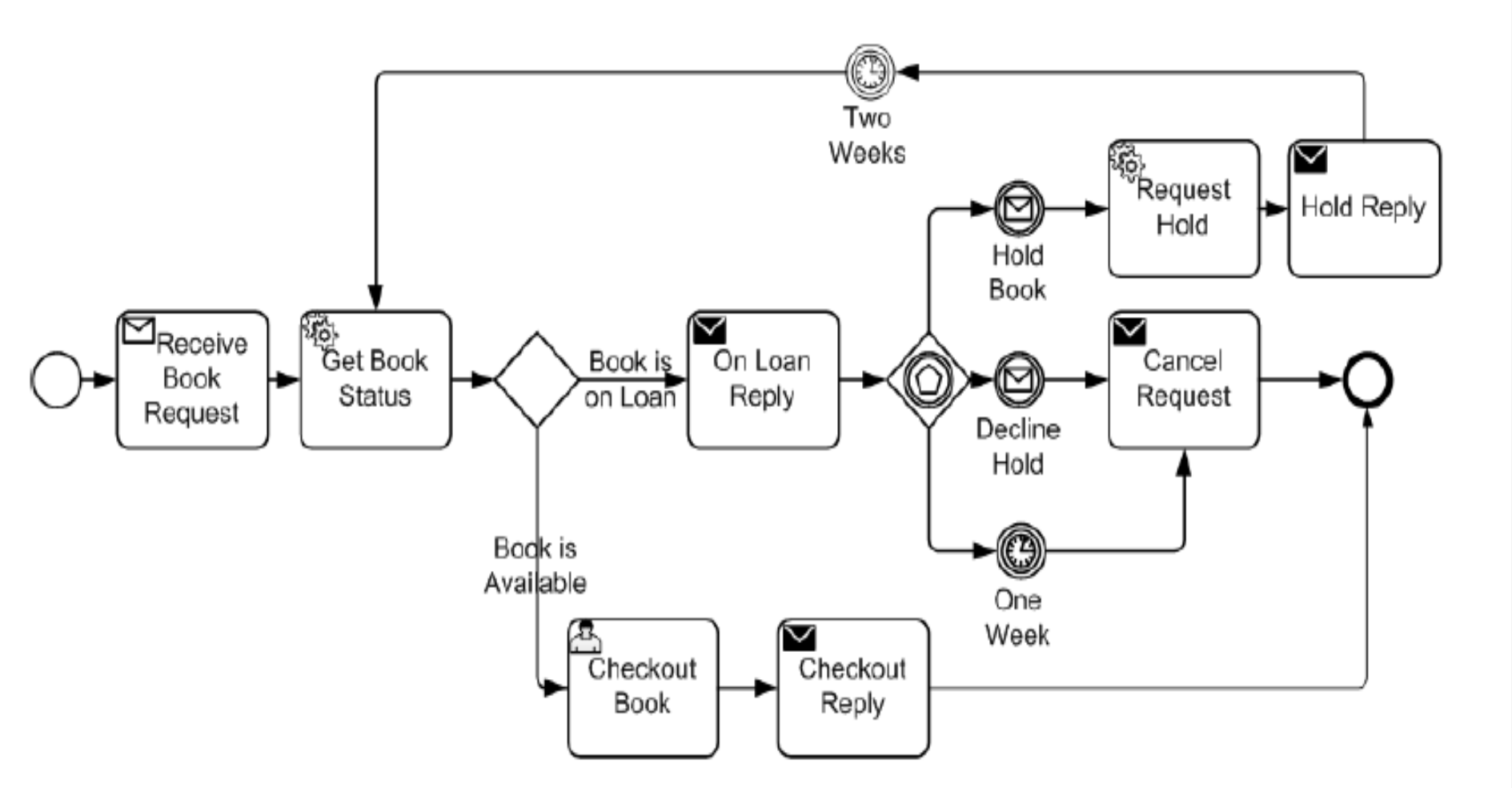
BPM for Ordering request



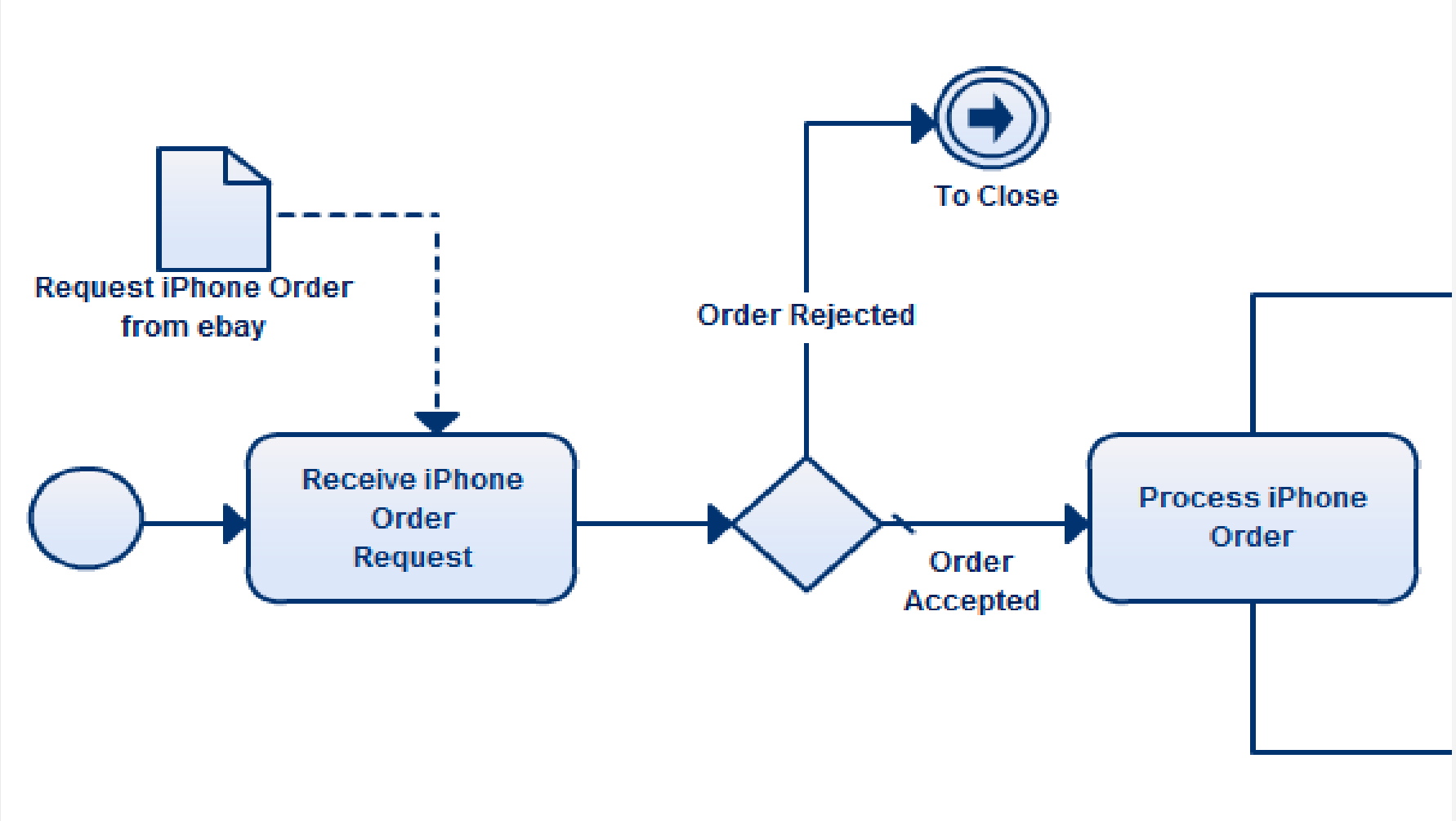
A Business Process Modeled Using BPN Notation



Business Process Model for Book Requisition Request



Business Process Model for product order from ebay



Why Use Business Process Modeling

1. Create process repetition
2. Improve Process Communication
3. Remove redundant tasks
4. Create transparency
5. Adds flexibility

References

1. **Software Engineering A practitioner's Approach** by Roger S. Pressman, 7th edition, McGraw Hill, 2010.
2. **Software Engineering by Ian Sommerville,** 9th edition, Addison-Wesley, 2011
3. **Systems Analysis and Design,** Kendall and Kendall, Fifth Edition
4. https://www.lucidchart.com/pages/bpmn
5. https://creately.com/blog/diagrams/business-process-modeling-techniques/

9. Software Testing Strategies

Abdus Sattar
Assistant Professor
Department of Computer Science and Engineering
Daffodil International University
Email: abdus.cse@diu.edu.bd

Discussion Topics

- Program Testing
- Aim of Testing
- Verification vs Validation
- Design of Test Cases
- Functional Testing Vs. Structural Testing
- Black Box Testing
- White Box Testing

Program Testing

- Testing a program consists of providing the program with a set of test inputs (or test cases) and observing if the program behaves as expected. If the program fails to behave as expected, then the conditions under which failure occurs are noted for later debugging and correction.
- Some commonly used terms associated with testing are:
  - **Failure:** This is a manifestation of an error (or defect or bug). But, the mere presence of an error may not necessarily lead to a failure.
  - **Test case:** This is the triplet [I,S,O], where I is the data input to the system, S is the state of the system at which the data is input, and O is the expected output of the system.
  - **Test suite:** This is the set of all test cases with which a given software product is to be tested.

Aim of Testing

- The aim of the testing process is to identify all defects existing in a software product.
- However for most practical systems, even after satisfactorily carrying out the testing phase, it is not possible to guarantee that the software is error free. This is because of the fact that the input data domain of most software products is very large. It is not practical to test the software exhaustively with respect to each value that the input data may assume.
- Even with this practical limitation of the testing process, the importance of testing should not be underestimated.
- It must be remembered that testing does expose many defects existing in a software product. Thus testing provides a practical way of reducing defects in a system and increasing the users' confidence in a developed system.

Verification vs Validation

- **Verification** is the process of determining whether the output of one phase of software development conforms to that of its previous phase.
  - Verification is concerned with phase containment of errors.
- **Validation** is the process of determining whether a fully developed system conforms to its requirements specification.
  - Aim of validation is that the final product be error free.

Design of Test Cases

- Exhaustive testing of almost any non-trivial system is impractical due to the fact that the domain of input data values to most practical software systems is either extremely large or infinite.
- Therefore, we must design an optional test suite that is of reasonable size and can uncover as many errors existing in the system as possible.
- But larger test suite does not always detects more error. For example lets consider the following code:
  - **if (x>y)**
    max = x;
    else
    max = x;

Design of Test Cases(Cont..)

- For the above code segment, consider the following test suites
  - Test suite 1: **{ (x=3,y=2); (x=2,y=3) }**
  - Test suite 2: **{ (x=3,y=2); (x=4,y=3); (x=5,y=1) }**
- Test suite 1 can detect the error.
- Test suite 2 can not detect the error despite of being large.
- Therefore, systematic approaches should be followed to design an optimal test suite. In an optimal test suite, each test case is designed to detect different errors.

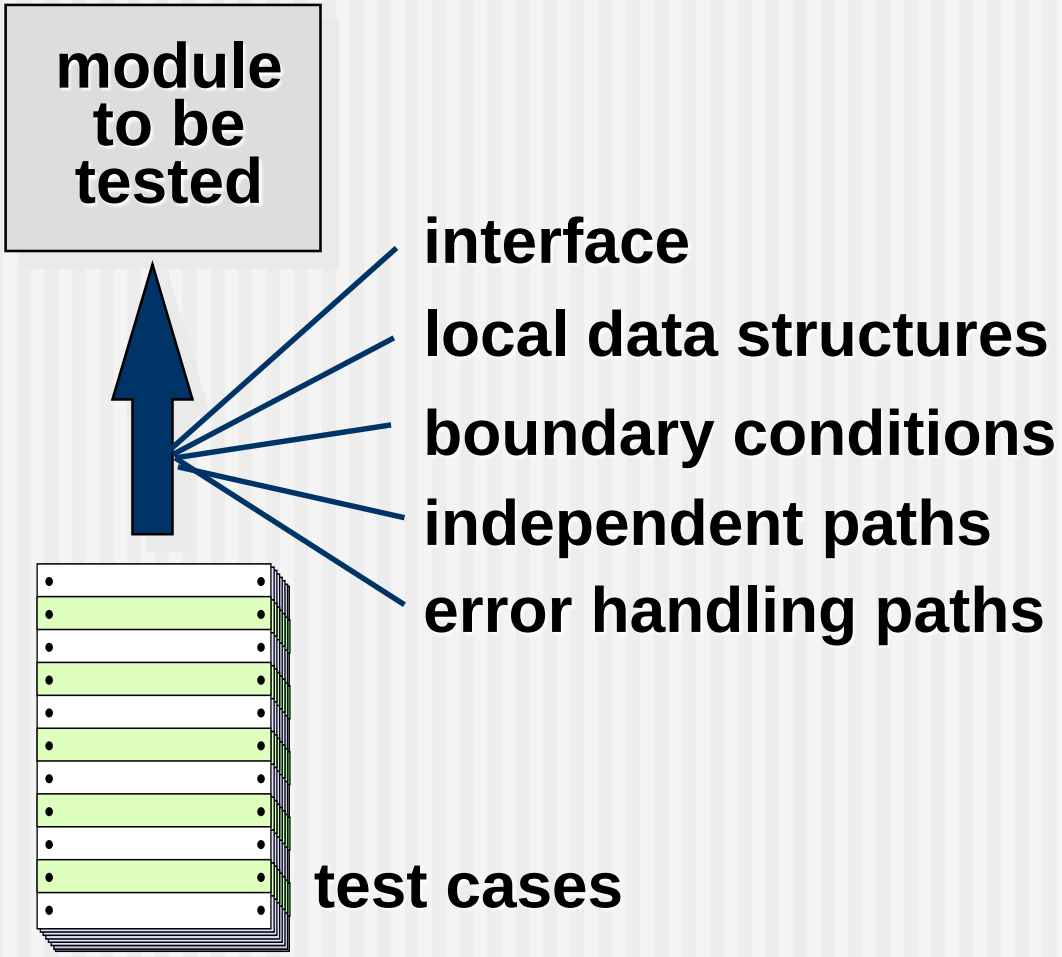
Functional Testing Vs. Structural Testing

- In the black-box testing approach, test cases are designed using only the functional specification of the software, i.e. without any knowledge of the internal structure of the software. For this reason, black-box testing is known as functional testing.
- On the other hand, in the white-box testing approach, designing test cases requires thorough knowledge about the internal structure of software, and therefore the white-box testing is called structural testing.

Unit testing

- Unit testing is undertaken after a module has been coded and successfully reviewed. Unit testing (or module testing) is the testing of different units (or modules) of a system in isolation.
- In order to test a single module, a complete environment is needed to provide all that is necessary for execution of the module. That is, besides the module under test itself, the following steps are needed in order to be able to test the module:
  - The procedures belonging to other modules that the module under test calls.
  - Nonlocal data structures that the module accesses.
  - A procedure to call the functions of the module under test with appropriate parameters.

Unit Testing



Black Box Testing

- In the black-box testing
  - test cases are designed from an examination of the input/output values only
  - no knowledge of design or code is required.
- The following are the two main approaches to designing black box test cases.
  - Equivalence class portioning
  - Boundary value analysis

Black Box Testing

- **In Black Box Testing we just focus on inputs and output of the software system** without bothering about internal knowledge of the software program.

Equivalence Class Partitioning

- In this approach, the domain of input values to a program is partitioned into a set of equivalence classes.
- The following are some general guidelines for designing the equivalence classes:
  - If the input data values to a system can be specified by a range of values, then one valid and two invalid equivalence classes should be defined.
  - If the input data assumes values from a set of discrete members of some domain, then one equivalence class for valid input values and another equivalence class for invalid input values should be defined.

Equivalence Class Partitioning

- **Example 1:** A software can compute the square root of an input integer which can assume values in the range of 0 to 5000. Design 3 Equivalence Class Partitioning test cases.
- **Answer:** There are three equivalence classes:
  - The set of negative integers.
  - the set of integers in the range of 0 and 5000 and
  - the integers larger than 5000.
  - Therefore, the test cases must include representatives for each of the three equivalence classes and possible 3 test sets can be:
  - Test suite 1: {-5,500,6000}, Test suite 2: {-15,900,6020} and Test suite 3: {-3,4999,5100}.

Equivalence Class Partitioning

- **Example 2:** Design the black-box test suite for the following program. The program computes the intersection point of two straight lines and displays the result. It reads two integer pairs (m1, c1) and (m2, c2) defining the two straight lines of the form y=mx + c.
- **Answer :** The equivalence classes are the following:
  - Parallel lines (m1=m2, c1≠c2)
  - Intersecting lines (m1≠m2)
  - Coincident lines (m1=m2, c1=c2)
  - Now, selecting one representative value from each equivalence class, the test suit {(2, 2) (2, 5)}, {(5, 5) (7, 7)}, {(10, 10) (10, 10)} are obtained.

Boundary Value Analysis

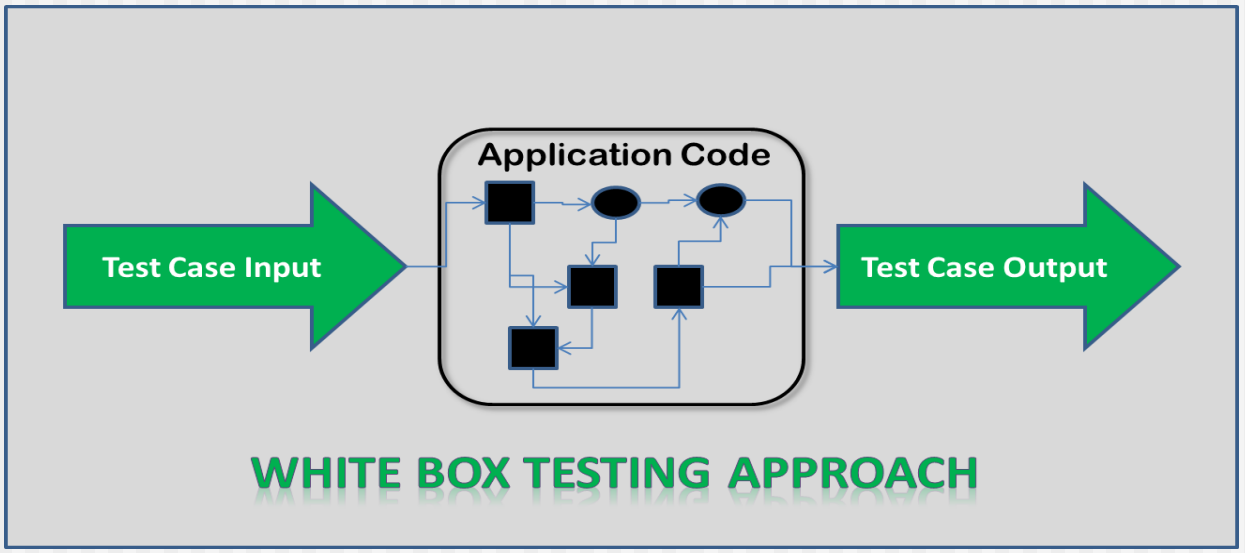
- A type of programming error frequently occurs at the boundaries of different equivalence classes of inputs. The reason behind such errors might purely be due to psychological factors. Programmers often fail to see the special processing required by the input values that lie at the boundary of the different equivalence classes. For example, programmers may improperly use < instead of <=, or conversely <= for <. Boundary value analysis leads to selection of test cases at the boundaries of the different equivalence classes.
- **Example:** For a function that computes the square root of integer values in the range of 0 and 5000, the test cases must include the following values: {0, -1,5000,5001}.

WHITE-BOX TESTING

- WHITE BOX TESTING (also known as Clear Box Testing, Open Box Testing, Glass Box Testing, Transparent Box Testing, Code-Based Testing or Structural Testing) is a software testing method in which the internal structure/design/implementation of the item being tested is known to the tester.
- One white-box testing strategy is said to be *stronger than* another strategy, if all types of errors detected by the first testing strategy is also detected by the second testing strategy, and the second testing strategy additionally detects some more types of errors. When two testing strategies detect errors that are different at least with respect to some types of errors, then they are called *complementary*.

White Box Testing

- White-box testing (also known as clear box testing, glass box testing, transparent box testing, and structural testing) is a method of testing software that tests internal structures or workings of an application, as opposed to its functionality



WHITE-BOX TESTING

- Example: Consider the Euclid's GCD computation algorithm:

```
int compute_gcd(x, y) {
        int x, y;
        while (x! = y) {
                if (x>y)
                    x= x – y;
                else
                    y= y – x;
                }
        return x;
}
```

- By choosing the test set {(x=3, y=3), (x=4, y=3), (x=3, y=4)}, we can exercise the program such that all statements are executed at least once.

References

Software Maintenance Models

https://www.professionalqa.com/software-maintenance-models

Chapter 9 software maintenance

https://www.slideshare.net/abhinavtheneo/chapter-9-software-maintenance

10. Software Maintenance and Maintenance Process Model

CSE333: Software Engineering

Abdus Sattar
Assistant Professor
Department of Computer Science and Engineering
Daffodil International University
Email: abdus.cse@diu.edu.bd

Learning Outcomes

- Understand the stages of testing from testing, during development to acceptance testing by system customers.
- have been introduced to techniques that help you choose test cases that are geared to discovering program defects.
- Understand test-first development, where you design tests before writing code and run these tests automatically.

Software Maintenance

- Software Maintenance is the process of modifying a software product after it has been delivered to the customer.
- The main purpose of software maintenance is to modify and update software application after delivery to correct faults and to improve performance.
- The purpose of **software maintenance** is to preserve the value of software over time, which can accomplished by:
  - Expanding the customer base.
  - Enhancing software's capabilities.
  - Omitting obsolete capabilities.
  - Employing newer technology.

Categories Software Maintenance

- Basic software maintenance includes optimization, error correction, and enhancement of existing features, which combine together to make the software abreast with the latest changes and demands of the software industry.

1. **Corrective Maintenance:**
   - Corrective Maintenance is a reactive process that is **focused on fixing failures in the system.**
   - It refers to the modification and enhancement done to the coding and design of a software to fix errors or defects detected by the user or concluded by error user report.
   - Corrective Maintenance is further divided into two types:
     - **Emergency Repairs.**
     - **Scheduled Repairs.**

Categories Software Maintenance(Cont..)

2. Adaptive Maintenance: Adaptive Maintenance is initiated as a consequence of internal needs, like moving the software to a different hardware or software platform compiler, operating system or new processor and to match the external completion and requirements.

3. Perfective Maintenances: The process of perfective maintenance includes making the product faster, cleaner structured, improving its reliability and performance, adding new features, and more.

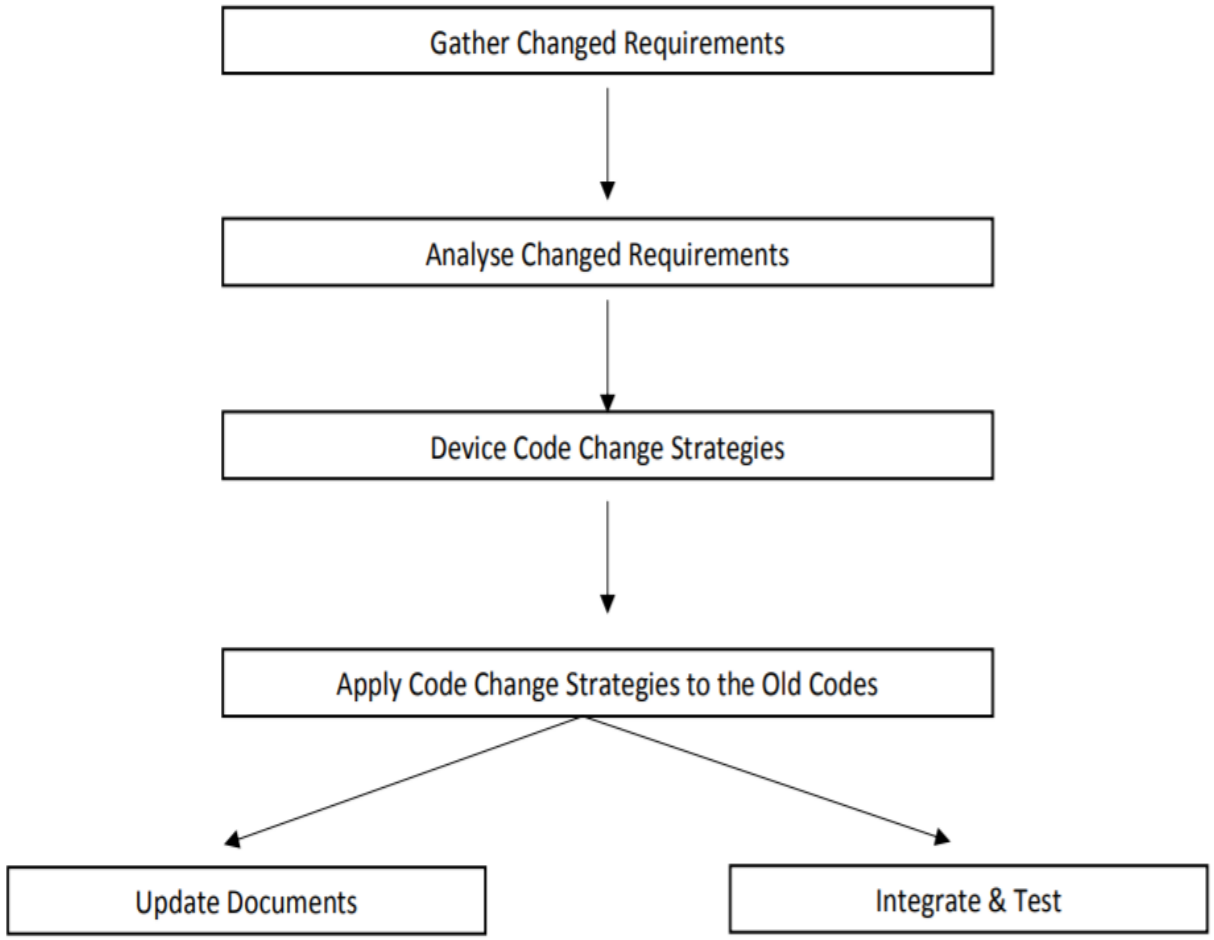
4. Preventive Maintenance: The objective of Preventive Maintenance is to attend problems, which may seem insignificant but can cause serious issues in the future.

Maintenance Process Models

- Two broad categories of process models for software maintenance can be proposed.
- The first model is preferred for projects involving small reworks where the code is changed directly and the changes are reflected in the relevant documents later. This maintenance process is graphically presented **Process Model-1**.
- The second process model for software maintenance is preferred for projects where the amount of rework required is significant. This maintenance process is graphically presented in Process Model-2.

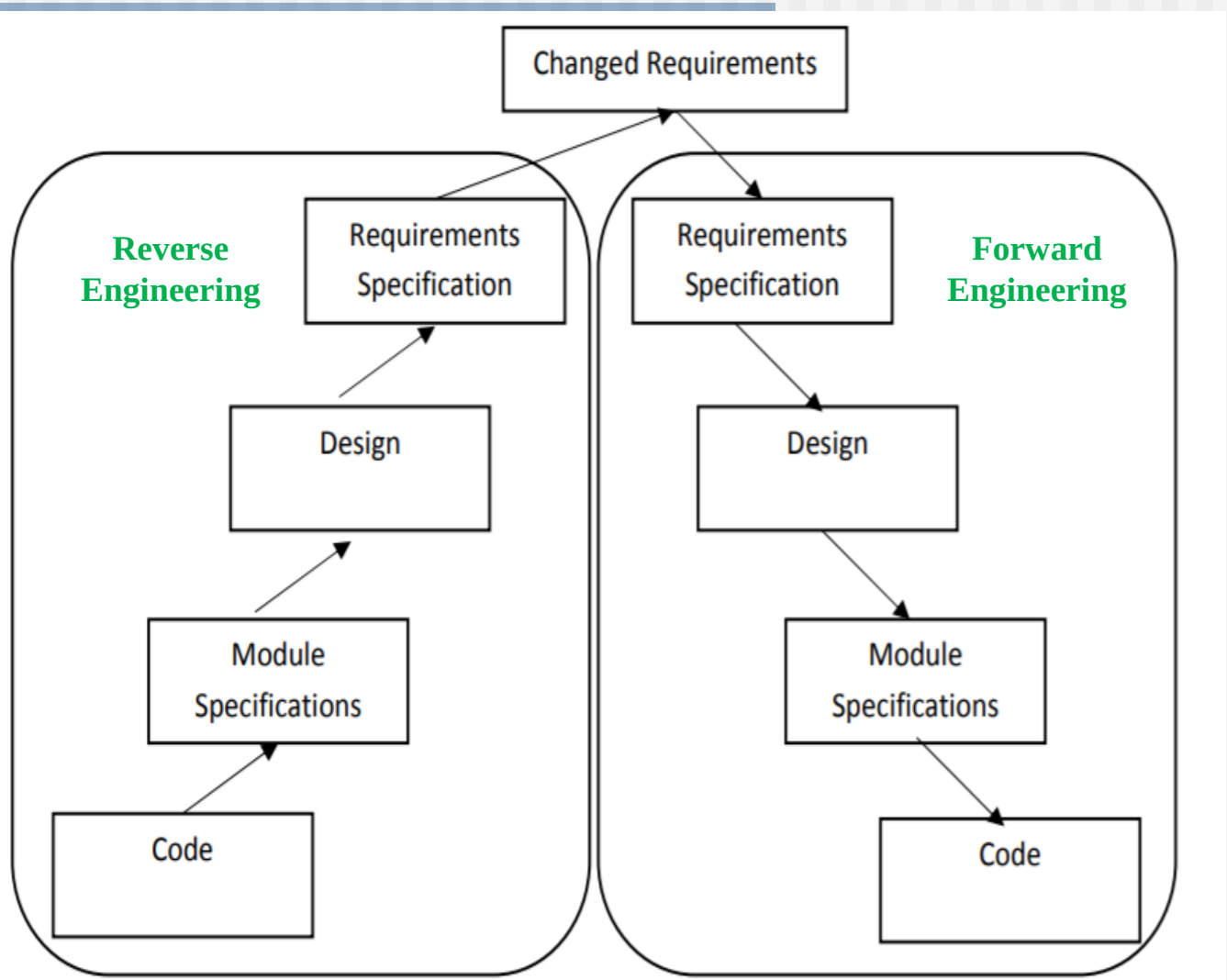
Maintenance Process Model 1

- In this approach, the project starts by gathering the requirements for changes. The requirements are next analyzed to formulate the strategies to be adopted for code change.
- At this stage, the association of at least a few members of the original development team goes a long way in reducing the cycle team, especially for projects involving unstructured and inadequately documented code.
- The availability of a working old system to the maintenance engineers at the maintenance site greatly facilitates the task of the maintenance team as they get a good insight into the working of the old system and also can compare the working of their modified system with the old system.
- Also, debugging o f the reengineered system becomes easier as the program traces of both the systems can be compared to localize the bugs.



Maintenance Process Model 2

- The second process model for software maintenance is preferred for projects where the amount of rework required is significant.
- This approach can be represented by a reverse engineering cycle followed by a forward engineering cycle. Such an approach is also known as software reengineering.
- During the reverse engineering, the old code is analyzed to extract the module specifications. The module specifications are then analyzed to produce the design. The design is analyzed to produce the original requirements specification. The change requests are then applied to this requirements specification to arrive at the new requirements specification. At the design, module specification, and coding a substantial reuse is made from the reverse engineered products



Estimation of approximate Maintenance Cost

- It is well known that maintenance efforts require about 60% of the total life cycle cost for a typical software product.
- However, maintenance costs vary widely from one application domain to another.
- Boehm [1981] proposed a formula for estimating maintenance costs as part of his COCOMO cost estimation model. Boehm's maintenance cost estimation is made in terms of a quantity called the Annual Change Traffic (ACT).

Estimation of Approximate Maintenance Cost

Boehm's maintenance cost estimation is made in terms of a quantity called the **Annual Change Traffic (ACT).** Boehm defined ACT as the fraction of a software product's source instructions which undergo change during a typical year either through addition or deletion.

$$ACT = \frac{KLOC_{added} + KLOC_{deleted}}{KLOC_{total}}$$

where, KLOC added is the total kilo lines of source code added during maintenance.
KLOC deleted is the total kilo lines of source code deleted during maintenance.

Thus, the code that is changed, should be counted in both the code added and the code deleted. The annual change traffic (ACT) is multiplied with the total development cost to arrive at the maintenance cost:

$$\text{maintenance cost} = ACT \times \text{development cost}.$$

Exercise

Find the maintenance cost of a software product where there are a total of 55000 lines of coding. During maintenance 80000 lines of coding was added and 3500 lines of coding was deleted. Development cost of the project was 5 lacs BDT.

References

1. **Software Engineering A practitioner's Approach** by Roger S. Pressman, 7th edition, McGraw Hill, 2010.

2. **Lecture Notes on:** https://docplayer.net/8258886-Lecture-notes-on-software-engineering-course-code-bcs-306.html

11. COCOMO Model

CSE333: Software Engineering

Abdus Sattar
Assistant Professor
Department of Computer Science and Engineering
Daffodil International University
Email: abdus.cse@diu.edu.bd

Discussion Agenda

- **Boehm's Definition of Software Project Types**
- **COCOMO**
- **Basic COCOMO**
- **Estimation of development effort**
- **Estimation of development time**
- **Example practicing**

Boehm's Definition of Software Project Types

- Boehm postulated that any software development project can be classified into one of the following three categories based on the development complexity
  - Organic
  - Semidetached
  - Embedded.

Boehm's Definition

- Boehm's [1981] definition of organic, semidetached, and embedded systems are elaborated below.
  - **Organic:** A development project can be considered of organic type, if the project deals with developing a well understood application program, the size of the development team is reasonably small, and the team members are experienced in developing similar types of projects.
  - **Semidetached:** A development project can be considered of semidetached type, if the development consists of a mixture of experienced and inexperienced staff. Team members may have limited experience on related systems but may be unfamiliar with some aspects of the system being developed.
  - **Embedded:** A development project is considered to be of embedded type, if the software being developed is strongly coupled to complex hardware, or if the stringent regulations on the operational procedures exist.

COCOMO

- COCOMO (Constructive Cost Estimation Model) was proposed by Boehm [1981].
- According to Boehm, software cost estimation should be done through three stages
  - Basic COCOMO
  - Intermediate COCOMO
  - Complete COCOMO.

Basic COCOMO

- The basic COCOMO model gives an approximate estimate of the project parameters. The basic COCOMO estimation model is given by the following expressions:

$$\text{Effort} = a_1 \times (\text{KLOC})^{a_2} \text{ PM}$$

$$\text{Tdev} = b_1 \times (\text{Effort})^{b_2} \text{ Months}$$

- Here
  - **KLOC** is the estimated size of the software product expressed in Kilo Lines of Code,
  - a_1, a_2, b_1, b_2 are constants for each category of software products,
  - **Tdev** is the estimated time to develop the software, expressed in months,
  - Effort is the total effort required to develop the software product, expressed in person months (PMs).

Estimation of Development Effort

- For the three classes of software products, the formulas for estimating the effort based on the code size are shown below:

Organic : $\mathbf{Effort} = 2.4(KLOC)^{1.05}$ PM

Semi-detached : $\mathbf{Effort} = 3.0(KLOC)^{1.12}$ PM

Embedded : $\mathbf{Effort} = 3.6(KLOC)^{1.20}$ PM

Estimation of Development Time

- For the three classes of software products, the formulas for estimating the development time based on the effort are given below:

Organic : $\mathbf{Tdev = 2.5(Effort)^{0.38}\ Months}$

Semi-detached : $\mathbf{Tdev = 2.5(Effort)^{0.35}\ Months}$

Embedded : $\mathbf{Tdev = 2.5(Effort)^{0.32}\ Months}$

Example

- Q. Assume that the size of an organic type software product has been estimated to be 32,000 lines of source code. Assume that the average salary of software engineers be Rs. 15,000/- per month. Determine the effort required to develop the software product and the nominal development time.

- **Answer:** From the basic COCOMO estimation formula for organic software:
  - Effort = 2.4 x $(32)^{1.05}$ = 91 PM
  - Nominal development time = 2.5 x $(91)^{0.38}$ = 14 months
  - Cost required to develop the product = 14 x 15,000 = 210,000/=

References

1. **Software Engineering A practitioner's Approach** by Roger S. Pressman, 7th edition, McGraw Hill, 2010.

2. **COCOMO Model:** https://www.educba.com/cocomo-model/

3. **Software Engineering COCOMO Model:** https://www.geeksforgeeks.org/software-engineering-cocomo-model/

12. Software Quality Assurance and Quality Management

CSE333: Software Engineering

Abdus Sattar

Assistant Professor

Department of Computer Science and Engineering

Daffodil International University

Email: abdus.cse@diu.edu.bd

Daffodil International University

Discussion Topics

- Software Quality Assurance
- What are SQA, SQP, SQC, and SQM?
- Elements of Software Quality Assurance
- SQA Tasks, Goals, And Metrics
- Quality Management and Software Development
- QA Vs QC
- Reviews and Inspections
- An Inspection Checklist

Software Quality Assurance

- **Software Quality Assurance** (SQA) is a set of activities for ensuring quality in software engineering processes. It ensures that developed software meets and complies with the defined or standardized quality specifications.
- SQA is an ongoing process within the Software Development Life Cycle (SDLC) that routinely checks the developed software to ensure it meets the desired quality measures.

What are SQA, SQP, SQC, and SQM?

- **Software Quality Assurance** – establishment of network of organizational procedures and standards leading to high-quality software
- **Software Quality Planning** – selection of appropriate procedures and standards from this framework and adaptation of these to specific software project
- **Software Quality Control** – definition and performing of processes that ensure that project quality procedures and standards are being followed by the software development team
- **Software Quality Metrics** – collecting and analyzing quality data to predict and control quality of the software product being developed

Elements of Software Quality Assurance

- **Standards.** The IEEE, ISO, and other standards organizations have produced a broad array of software engineering standards and related documents.
- **Reviews and audits.** Technical reviews are a quality control activity performed by software engineers for software engineers.
- **Testing.** Software testing is a quality control function that has one primary goal—to find errors.
- **Error/defect collection and analysis.** The only way to improve is to measure how you're doing. SQA collects and analyzes error and defect data to better understand how errors are introduced and what software engineering activities are best suited to eliminating them.
- **Change management.** Change is one of the most disruptive aspects of any software project. If it is not properly managed, change can lead to confusion, and confusion almost always leads to poor quality.
- **Education.** Every software organization wants to improve its software engineering practices. A key contributor to improvement is education of software engineers, their managers, and other stakeholders.

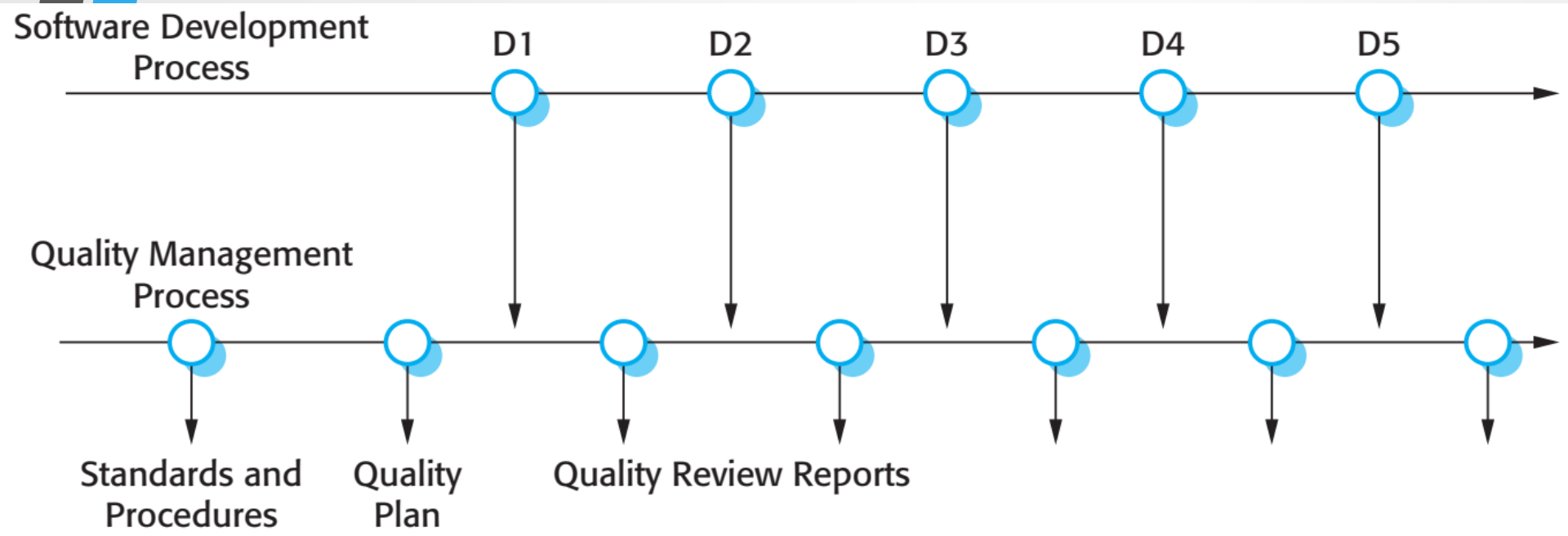
Elements of Software Quality Assurance(Cont…)

- **Vendor management.** Three categories of software are acquired from external software vendors—*shrink-wrapped packages* (e.g., Microsoft *Office*), that is custom tailored to the needs of a purchaser, and *contracted software* that is custom designed and constructed from specifications provided by the customer organization.
- **Security management.** With the increase in cyber crime and new government regulations regarding privacy, every software organization should institute policies that protect data at all levels, establish firewall protection for WebApps, and ensure that software has not been tampered with internally. SQA ensures that appropriate process and technology are used to achieve software security.
- **Safety.** SQA may be responsible for assessing the impact of software failure and for initiating those steps required to reduce risk.
- **Risk management.** Risk management activities are properly conducted and that risk-related contingency plans have been established

SQA Tasks, Goals, And Metrics

Goal	Attribute	Metric
Requirement quality	Ambiguity	Number of ambiguous modifiers (e.g., many, large, human-friendly)
	Completeness	Number of TBA, TBD
	Understandability	Number of sections/subsections
	Volatility	Number of changes per requirement
		Time (by activity) when change is requested
	Traceability	Number of requirements not traceable to design/code
	Model clarity	Number of UML models
		Number of descriptive pages per model
		Number of UML errors
Design quality	Architectural integrity	Existence of architectural model
	Component completeness	Number of components that trace to architectural model
		Complexity of procedural design
	Interface complexity	Average number of pick to get to a typical function or content
		Layout appropriateness
	Patterns	Number of patterns used
Code quality	Complexity	Cyclomatic complexity
	Maintainability	Design factors (Chapter 8)
	Understandability	Percent internal comments
		Variable naming conventions
	Reusability	Percent reused components
	Documentation	Readability index
QC effectiveness	Resource allocation	Staff hour percentage per activity
	Completion rate	Actual vs. budgeted completion time
	Review effectiveness	See review metrics (Chapter 14)
	Testing effectiveness	Number of errors found and criticality
		Effort required to correct an error
		Origin of error

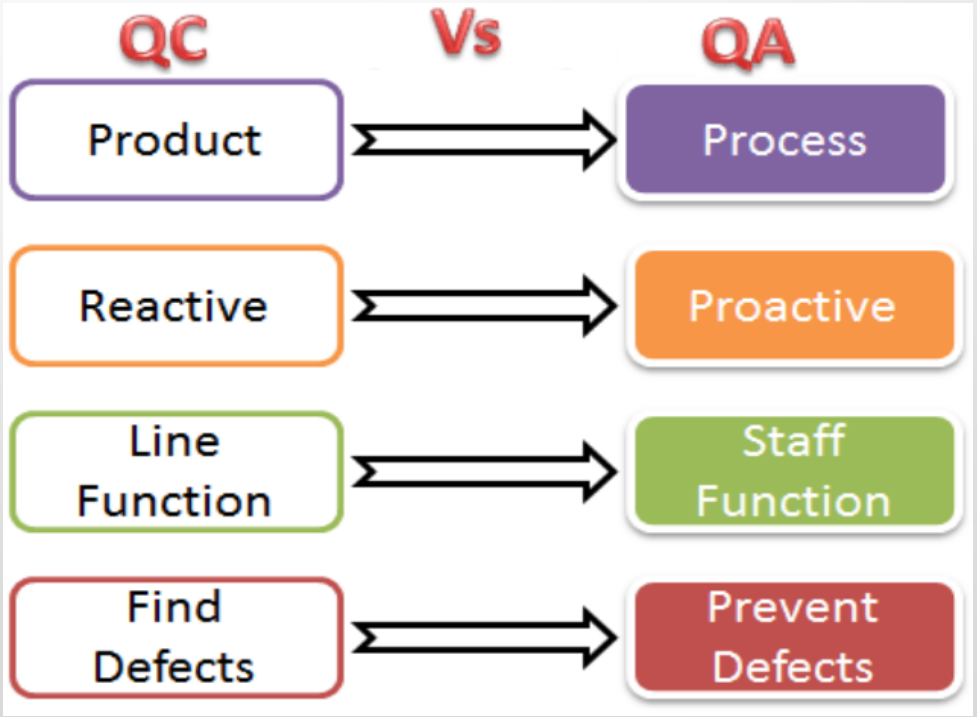
Quality Management and Software Development



- **Quality management** provides an independent check on the software development process.
- **The quality management** process checks the project deliverables to ensure that they are consistent with organizational standards and goals.
- QA team should be independent from the development team so that they can take an objective view of the software. This allows them to report on software quality without being influenced by software development issues.

Quality Management

- The terms ‘quality assurance’ and ‘quality control’ are widely used in manufacturing industry.
- **Quality Assurance (QA)** is the definition of processes and standards that should lead to high-quality products and the introduction of quality processes into the manufacturing process.
- **Quality Control(QC)** is the application of these quality processes to weed out products that are not of the required level of quality.



Reviews and Inspections

- **Reviews and inspections** are QA activities that check the quality of project deliverables. This involves examining the software, its documentation and records of the process to discover errors and omissions and to see if quality standards have been followed.
- **During a review**, a group of people examine the software and its associated documentation, looking for potential problems and non-conformance with standards. The review team makes informed judgments about the level of quality of a system or project deliverable. Project managers may then use these assessments to make planning decisions and allocate resources to the development process.
- **Program inspections** are 'peer reviews' where team members collaborate to find bugs in the program that is being developed.
- **Program inspections** involve team members from different backgrounds who make a careful, line-by-line review of the program source code. They look for defects and problems and describe these at an inspection meeting. Defects may be logical errors, anomalies in the code that might indicate an erroneous condition or features that have been omitted from the code. The review team examines the design models or the program code in detail and highlights anomalies and problems for repair.
- **During an inspection**, a checklist of common programming errors is often used to focus the search for bugs. This checklist may be based on examples from books or from knowledge of defects that are common in a particular application domain.

An Inspection Checklist

Fault class	Inspection check
Data faults	• Are all program variables initialized before their values are used? • Have all constants been named? • Should the upper bound of arrays be equal to the size of the array or Size -1? • If character strings are used, is a delimiter explicitly assigned? • Is there any possibility of buffer overflow?
Control faults	• For each conditional statement, is the condition correct? • Is each loop certain to terminate? • Are compound statements correctly bracketed? • In case statements, are all possible cases accounted for? • If a break is required after each case in case statements, has it been included?
Input/output faults	• Are all input variables used? • Are all output variables assigned a value before they are output? • Can unexpected inputs cause corruption?
Interface faults	• Do all function and method calls have the correct number of parameters? • Do formal and actual parameter types match? • Are the parameters in the right order? • If components access shared memory, do they have the same model of the shared memory structure?
Storage management faults	• If a linked structure is modified, have all links been correctly reassigned? • If dynamic storage is used, has space been allocated correctly? • Is space explicitly deallocated after it is no longer required?
Exception management faults	• Have all possible error conditions been taken into account?

❑ References:

1. **Software Engineering A practitioner's Approach**

   by Roger S. Pressman, 7th edition, McGraw Hill, 2010.

2. **Software Engineering by Ian Sommerville,**

   9th edition, Addison-Wesley, 2011

Ethical, Security, and Legal Issues in Software Engineering

Exploring the Implications for Organizations and Stakeholders

Introduction

- Information systems and business are closely intertwined
- Technology enables data collection, planning, and resource coordination
- Legal, security, and ethical issues arise from the use of information systems
- Implications affect both companies and their stakeholders

Photo by Pexels

Ethical Issues

- Privacy concerns with data collected by companies
- Accuracy challenges due to complex interactions between databases
- Data safeguarding and access control are crucial
- Ethical implications impact companies and individuals

Photo by Pexels

Legal Issues

- Violation of consumer rights through misuse of personal data
- Copyright infringement due to difficulty in classifying digital works
- Challenges in protecting intellectual property
- Legal implications pose risks for organizations and stakeholders

Photo by Pexels

Security Issues

- Threats to data security from unauthorized access and fraud
- Vulnerability to hackers and computer viruses
- Examples of system failures and their impacts
- Mitigating measures such as password protection and spam filtering

Photo by Pexels

Conclusion

- Ethical, security, and legal issues are significant in information systems
- Data privacy, accuracy, and protection are crucial
- Intellectual property and copyright are at risk
- Mitigating security threats is essential for organizations
- Addressing these issues is important for all stakeholders

Photo by Pexels